
COMPOSE: GENERATIVE STOCHASTIC REWRITING FOR TRANS-DIMENSIONAL MOLECULAR DESIGN AND CONTROL

Anonymous authors

Paper under double-blind review

ABSTRACT

Controllable molecular design requires both a representation of how molecules can change and a mechanism for steering those changes toward desired outcomes. Yet molecular generation and optimization are typically tied to fixed endpoint distributions or task objectives, rather than a reusable transition process that can be learned once and redirected as design goals change. We introduce COMPOSE (**C**ontrollable **M**olecular **P**rocess **O**ver **S**tochastic **E**ditions), a generative framework that represents molecular design as a stochastic sequence of executable edits between complete molecular graphs. An exact rewrite system defines legal, trans-dimensional molecular transitions, while a goal-independent reference process learns probability over them. This process is trained once and held fixed, and task-specific control steers the same molecular dynamics toward different objectives. On standard similarity-constrained QED editing on ZINC-250k, the same GuacaMol-trained process transfers without retraining and improves on recent size-adaptive graph diffusion results while using fewer returned trajectories. On protein-specific lead optimization under molecular feasibility and similarity constraints, COMPOSE improves paired docking performance over the published state of the art while using sixfold fewer docking-oracle evaluations and maintaining the same feasible coverage. Beyond these benchmarks, COMPOSE uses downstream reachability to guide multi-step decisions and supports intervention on realized molecular trajectories as objectives and constraints change. These results establish that a molecular transition process can be learned once and reused across datasets, objectives, and interventions without retraining the underlying generator.

1 INTRODUCTION

Generative modeling has become a central approach to molecular design, supporting both de novo generation and the optimization of existing molecules across increasingly complex chemical spaces (Gómez-Bombarelli et al., 2018; Jin et al., 2018; Vignac et al., 2023; Lee et al., 2025). Recent graph-based diffusion, flow, and discrete generative models can learn expressive molecular distributions and increasingly support conditional generation toward desired structural and functional properties (Vignac et al., 2023; Campbell et al., 2024; Lee et al., 2025). These advances have made molecular generation substantially more flexible, but the learned object is still typically an endpoint distribution or a generator specialized toward a particular task, rather than a reusable representation of transitions between molecular states. For molecular design, this suggests value in modeling the path between structures, since each transformation changes both the next molecule and the set of molecular states that remain reachable afterward.

Molecular editing makes this path dependence explicit, because each edit changes the transformations available from the resulting molecule. Existing editing and lead-optimization methods already exploit sequential molecular modifications through graph-to-graph generation, reinforcement learning, fragment-based search, evolutionary optimization, and MCMC (Jin et al., 2019; Zhou et al., 2019; Jensen, 2019; Xie et al., 2021; Fu et al., 2021). Yet the consequence of an edit extends beyond the molecule it immediately produces. An atom insertion, bond modification, or ring operation can open or eliminate entire downstream branches of the design space. Molecular editing may therefore benefit from representing not only the value of individual structures, but also which molecular futures remain reachable from each state.

Despite this progress, existing approaches capture only parts of a transition-centered formulation. Learned generators can increasingly be guided toward desired properties, but their generative trajectories are often defined through corrupted, masked, or latent states rather than reusable transitions between

complete molecules (Vignac et al., 2023; Campbell et al., 2024; Lee et al., 2025). Molecular editing and search methods instead operate directly on complete molecules, but their proposal mechanisms are typically prescribed by the optimizer and coupled to task-specific scoring (Jensen, 2019; Zhou et al., 2019; Xie et al., 2021). Recent work further shows that pretrained discrete generators can be redirected toward multiple objectives and constraints through inference-time guidance and control (Chen et al., 2025; 2026). Together, these advances motivate a complementary abstraction: a goal-independent generative process over molecular transitions that can be learned once and controlled as design goals change.

COMPOSE is related to variable-size discrete generation and editing, controlled stochastic generative processes, and multi-objective molecular optimization. GrIDDD and Edit Flows introduce variable-dimensional transitions (Ninniri et al., 2025; Havasi et al., 2025); MadSBM, Value Matching, and pCoMole use stochastic-process or value-based control (Goel and Chatterjee, 2026; Jensen et al., 2026; Chen et al., 2026); and methods such as AReURDi steer discrete generators toward molecular objectives (Chen et al., 2025). COMPOSE instead learns a goal-independent canonical transition law over an exact executable molecular rewrite space, then controls that same process using future molecular reachability (a full comparison is given in Appendix A).

We introduce **COMPOSE (Controllable Molecular Process Over Stochastic Edits)**, a generative framework that learns a reusable molecular transition process and controls it across design objectives (Figure 1). COMPOSE represents generation as a stochastic sequence of executable edits between complete molecules. At each molecular state, an exact executor defines what is possible by enumerating the state-dependent set of legal, trans-dimensional molecular successors. We then learn a goal-independent reference law R_θ over this same successor set from executable molecular trajectories, capturing which molecular continuations are plausible without conditioning on a downstream design goal. A separate finite-horizon controller determines what is purposeful by reweighting these molecular continuations according to the value of the futures they make reachable. Because generation and control operate on the same explicit molecular support, the transition process can be learned once and redirected across design objectives without retraining the underlying generator.

We summarize our main contributions as follows.

- We introduce a stochastic molecular process over exact, executable, trans-dimensional rewrites between complete molecular graphs, with a learned reference law defined on canonical molecular successors.
- We formulate molecular design as finite-horizon control of this frozen process, allowing edits to be valued by the molecular futures they leave reachable without introducing transitions outside the reference support.
- We show that the reference process learns state-dependent transition preferences, transfers across external molecular-design tasks without retraining, and supports future-aware editing, mid-trajectory retargeting, and pathwise constraints.

2 COMPOSE

COMPOSE represents molecular design as control of a learned stochastic process over complete molecules (Figure 1). An exact rewrite executor determines which molecular transitions are available, a goal-independent reference model learns probability over those transitions, and finite-horizon control reweights the frozen process toward a downstream design objective.

2.1 EXECUTABLE MOLECULAR STATE SPACE

Let

$$\mathcal{X} = \bigcup_{n=1}^{n_{\max}} \mathcal{X}_n$$

denote the molecular state space, where $\mathcal{X}_n$ contains connected molecular graphs with n heavy atoms, declared atom and bond attributes, and valid valences. Every $x \in \mathcal{X}$ is therefore a complete molecule rather than a masked, corrupted, or partially specified state.

Transitions are typed graph rewrites. An edit mark a specifies a rewrite family, molecular operands, and a typed payload, and $T(x, a)$ denotes the resulting graph. From state x , the executor exposes

$$\mathcal{A}(x) = \{a : a \text{ matches } x \text{ and } T(x, a) \text{ passes all executor guards}\}. \quad (1)$$

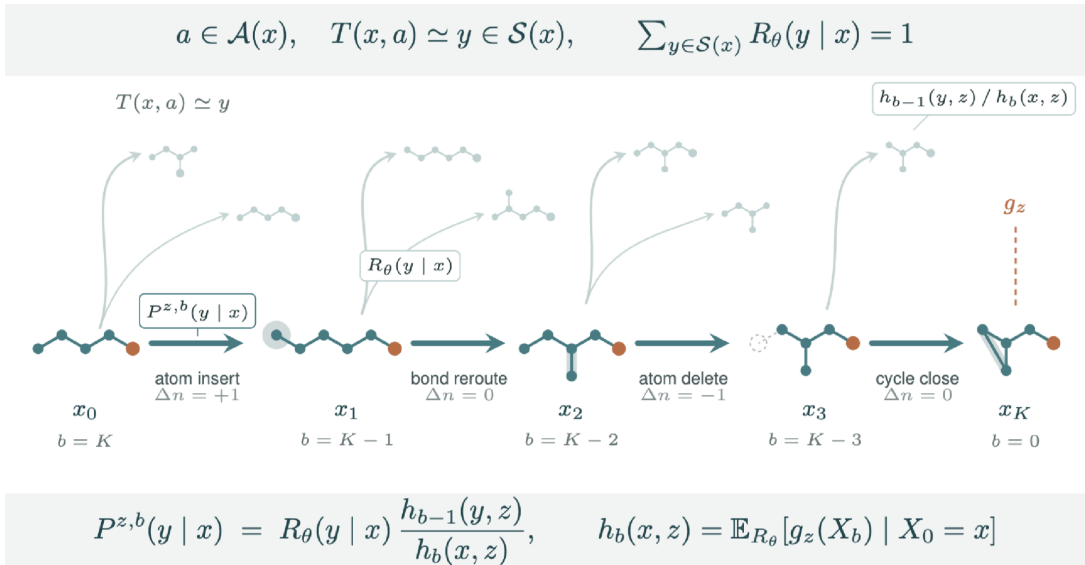


Figure 1: Overview of COMPOSE. From a molecule x , the graph-rewrite executor admits edit marks $a \in \mathcal{A}(x)$, which induce distinct canonical successors $y \in \mathcal{S}(x)$. The goal-independent reference kernel $R_\theta(y | x)$ assigns probability over this executable successor set, and iterating R_θ defines a trans-dimensional process over complete molecules. For objective z and b remaining edits, finite-horizon control reweights each reference transition by its remaining future value, $P^{z,b}(y | x) = R_\theta(y | x) h_{b-1}(y, z) / h_b(x, z)$, with $h_b(x, z) = \mathbb{E}_{R_\theta}[g_z(X_b) | X_0 = x]$. Control therefore changes which executable molecular transitions are preferred without introducing transitions outside the support of R_θ .

The guards enforce the declared atom and bond vocabularies, valence constraints, connectivity, and molecular-size bounds before an edit is committed. Inadmissible states therefore have zero transition support rather than receiving a learned penalty. The rewrite language includes atom insertion and deletion together with atom-, bond-, and ring-level transformations (Appendix F). Insertions and deletions connect $\mathcal{X}_n$ with $\mathcal{X}_{n \pm 1}$, allowing trajectories to grow, shrink, and restructure molecules within the same state space.

Because every admitted rewrite passes these guards, $T(x, a) \in \mathcal{X}$ for all $x \in \mathcal{X}$ and $a \in \mathcal{A}(x)$; the formal closure statement and full rewrite semantics are given in Appendix F. Multi-edit ring construction and refinement used in molecular optimization compile into the same primitive rewrite language and do not enlarge this support (Appendix F.3).

The edit sets $\mathcal{A}(x)$ specify the support of the process. We next learn how probability is distributed across it.

2.2 LEARNING THE REFERENCE PROCESS

We learn a goal-independent stochastic law over executable edits. Writing a mark as $a = (k, o, \eta)$, with rewrite family k , operands o , and typed payload η , the reference model factorizes

$$p_\theta(a | x) = p_\theta(k | x) p_\theta(o, \eta | k, x), \quad a \in \mathcal{A}(x). \quad (2)$$

Normalization is restricted to candidates admitted by the executor, and the model receives no design objective, remaining budget, or task-specific constraint. The learned law therefore depends on the current molecular state but not on the downstream purpose for which the process will later be used.

The stochastic object used for control is a distribution over molecular successors, not raw edit marks. Different marks can produce the same molecule because of graph symmetries, equivalent matches, or alternative internal rewrite descriptions. For a canonical successor y , define its executable fiber

$$\mathcal{G}_y(x) = \{a \in \mathcal{A}(x) : T(x, a) \simeq y\},$$

where $\simeq$ denotes registered molecular canonicalization. Let $\mathcal{S}(x)$ denote the distinct non-self canonical successors of x . Conditioning the pushforward mark law on a committed change of molecular state gives

$$R_\theta(y | x) = \frac{\sum_{a \in \mathcal{G}_y(x)} p_\theta(a | x)}{\sum_{y' \in \mathcal{S}(x)} \sum_{a \in \mathcal{G}_{y'}(x)} p_\theta(a | x)}, \quad y \in \mathcal{S}(x). \quad (3)$$

Aggregating over the fiber prevents a molecular transition from receiving additional probability merely because symmetry or implementation details provide several equivalent edit descriptions. Control therefore acts on R_θ after canonicalization rather than on individual edit marks.

We train R_θ from executable teacher transitions by minimizing the canonical-successor negative log-likelihood over transitions that commit a molecular jump,

$$\mathcal{L}_{\text{ref}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log R_\theta(y_i^* | x_i), \quad (4)$$

with rows sampled from the declared editing measure. The implementation carries a per-row registered transition weight, which is unity for every row of the frozen checkpoint reported here. Every teacher transition is replayed through the same executor used at inference. Targets and compiled programs are used to construct supervision but are not supplied to the reference model at inference. Once trained, R_θ is fixed across downstream objectives.

2.3 FINITE-HORIZON CONTROL OF THE FROZEN PROCESS

Task information enters only after the reference process has been learned. Let z denote a design objective and $g_z : \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}$ its terminal desirability. For b remaining transitions, define

$$h_0(x, z) = g_z(x), \quad h_b(x, z) = \sum_{y \in \mathcal{S}(x)} R_\theta(y | x) h_{b-1}(y, z) = \mathbb{E}_{R_\theta}[g_z(X_b) | X_0 = x]. \quad (5)$$

A property model scores the current molecule, whereas $h_b(x, z)$ measures the desirability reachable from that molecule within the remaining budget.

For $h_b(x, z) > 0$, the finite-horizon Doob transform defines

$$P^{z,b}(y | x) = R_\theta(y | x) \frac{h_{b-1}(y, z)}{h_b(x, z)}. \quad (6)$$

The controlled kernel reweights only transitions already supported by R_θ .

Proposition 1 (Finite-horizon molecular control). *Starting from x_0 with budget K , applying Equation (6) at each step yields*

$$\Pr_{P^z}(X_K = y | X_0 = x_0) = \Pr_{R_\theta}(X_K = y | X_0 = x_0) \frac{g_z(y)}{h_K(x_0, z)}.$$

Exact backward values therefore realize the g_z -tilted terminal law without introducing transitions outside the reference support. The proof and terminal-state assumptions are given in Appendix G.3.

Exact backward evaluation is infeasible in realistic molecular spaces. We therefore parameterize a nonnegative, goal- and budget-conditioned approximation $h_\phi(x, z, b) \approx h_b(x, z)$, learned from finite-budget continuations of the frozen reference process, and substitute it into Equation (6). Approximation error can alter probability among supported successors, but cannot introduce transitions outside R_θ . Exact terminal reweighting is therefore a property of the exact h -transform rather than the learned approximation. Training and sampling details are given in Appendices I and J.

Because control is recomputed from the molecule actually reached, its objective can change during a trajectory. If the process reaches x_τ under z and the objective changes to z' , control resumes from x_τ under z' with R_θ unchanged. We evaluate this setting in Section 3.3.

3 EXPERIMENTS

Our experiments address three questions. First, we ask whether the frozen reference law learns state-dependent preferences among executable molecular edits. Second, we test whether a reference process learned once on GuacaMol transfers unchanged to externally defined molecular-design benchmarks. Finally, we study trajectory-level capabilities that endpoint benchmarks do not capture: using downstream reachability to guide molecular edits, retargeting realized molecular states when objectives change, and enforcing constraints throughout a trajectory.

3.1 THE REFERENCE PROCESS LEARNS STATE-DEPENDENT EDIT PREFERENCES

The rewrite system specifies which molecular edits are executable from a given state, but it does not determine how probability should be distributed among them. We therefore test whether the frozen

reference law R_θ learns state-dependent transition preferences beyond the global frequencies of different edit families.

We evaluate 3,545 held-out transitions whose source molecules are disjoint from reference-law training. As a matched baseline, we assign probability according to empirical edit-family frequencies from the training set while preserving the same executable support and canonical successor set as R_θ . The comparison therefore isolates how probability is allocated among the molecular transitions available from each state.

R_θ reduces canonical-successor NLL from 5.37 to 3.90 nats relative to this empirical-family baseline. A matched decomposition shows that 86.7% of the improvement comes from learning which successor is appropriate for the current molecule, rather than simply learning which edit families are common overall (Table 6). The learned signal is therefore primarily state specific: the executor supplies the available molecular transitions, while R_θ learns how preference should be distributed across them.

We keep R_θ fixed in all subsequent experiments and use it as a goal-independent reference process for molecular design. Its transition probabilities are not intended to represent physical kinetics or synthetic-route likelihoods. Results restricted to edit families with data-backed transition supervision are reported in Appendix K.1.

3.2 A MOLECULAR PROCESS LEARNED ONCE TRANSFERS TO EXTERNAL DESIGN TASKS

We next evaluate whether the molecular process learned on GuacaMol can be reused without retraining R_θ . We keep the reference process and executable rewrite system fixed and apply COMPOSE to two external benchmarks with different design objectives: similarity-constrained QED editing on ZINC-250k, where molecules can grow or shrink during editing, and protein-specific lead optimization, where molecular properties define feasibility and docking score provides the optimization objective. Only the task-specific controller changes between these settings.

Similarity-constrained QED editing. We first consider the ZINC-250k editing benchmark used by GrIDDD (Ninniri et al., 2025). Source molecules have QED in $[0.7, 0.8]$, and a source is counted as solved if at least one of K returned molecules reaches $\text{QED} \geq 0.9$ while maintaining Morgan-fingerprint Tanimoto similarity ≥ 0.4 to the source. GrIDDD is a particularly relevant comparator because its generative process can insert and delete nodes, allowing molecular size to change during editing. Its reported model is trained directly on ZINC-250k.

For COMPOSE, the GuacaMol-trained R_θ from Section 3.1 is left unchanged. A QED-specific controller is fit on development sources disjoint from evaluation and acts on the same executable molecular process. Each of the K returned candidates corresponds to one controlled trajectory. We count a trajectory as successful when it first reaches the target region, and do not retrospectively add intermediate molecules to the candidate set.

On the official 800-source panel, COMPOSE solves 446 of the 800 sources (55.8%, 95% CI $[52.3, 59.2]$) with $K = 8$, compared with the 45.1% success reported by GrIDDD with $K = 20$ (Table 2). COMPOSE therefore solves more sources while returning fewer than half as many candidate trajectories per source. Among solved sources with at least two distinct non-source candidates, the mean within-source pairwise Tanimoto distance is 0.4999.

Similarity-constrained lead optimization. We next evaluate the same frozen reference process on the GenMol T4 benchmark (Lee et al., 2025), which combines molecular feasibility with a protein-specific optimization objective. The benchmark contains five protein targets, three seed molecules per target, and similarity thresholds $\delta \in \{0.4, 0.6\}$. Returned molecules must satisfy $\text{QED} \geq 0.6$, $\text{SA} \leq 4$, and Morgan-fingerprint Tanimoto similarity $\geq \delta$ to the seed. Among molecules satisfying these constraints, performance is measured by QuickVina2 docking score, with lower scores indicating stronger predicted binding.

We retain the same GuacaMol-trained R_θ and executable rewrite system used above, replacing only the task-specific controller. Here QED, SA, and similarity determine whether a candidate is eligible, while docking provides the optimization signal. Because these feasibility checks are inexpensive, they are evaluated before the protein oracle and docking calls are reserved for eligible molecules.

COMPOSE returns a feasible molecule in 26 of the 30 benchmark settings, matching GenMol’s coverage (Table 1). Across the 23 settings where both methods return a feasible molecule, COMPOSE improves the paired mean docking score by 0.335 kcal/mol (95% CI 0.019–0.651 in favor of COMPOSE). This performance is obtained with 500 docking-oracle evaluations per setting, compared with 3,000

Target	Seed score	$\delta = 0.4$				$\delta = 0.6$			
		COMPOSE	GenMol	RetMol	GraphGA	COMPOSE	GenMol	RetMol	GraphGA
PARP1	-7.3	-10.7	-10.6	-9.0	-8.3	-11.1	-10.4	—	-8.6
	-7.8	-11.5	-11.0	-10.7	-8.9	-10.9	-9.7	—	-8.1
	-8.2	-11.6	-11.3	-10.9	—	-10.2	-9.2	—	—
FA7	-6.4	-10.2	-8.4	-8.0	-7.8	-8.7	-7.3	-7.6	-7.6
	-6.7	-8.9	-8.4	—	-8.2	-7.2	-7.6	—	-7.6
	-8.5	-9.3	—	—	—	-7.9	—	—	—
5HT1B	-4.5	-11.7	-12.9	-12.1	-11.7	-13.4	-12.1	—	-11.3
	-7.6	-11.2	-12.3	-9.0	-12.1	-11.1	-12.0	-10.0	-12.0
	-9.8	-11.4	-11.6	—	—	—	-10.5	—	—
BRAF	-9.3	—	-10.8	—	-9.8	—	—	—	—
	-9.4	-11.2	-10.8	-11.6	—	-10.6	-9.7	—	—
	-9.8	-10.9	-10.6	—	-11.6	—	-10.5	—	-10.4
JAK2	-7.7	-10.3	-10.2	-8.2	-8.7	-9.3	-9.3	—	-8.1
	-8.0	-9.8	-10.0	-9.0	-9.2	-9.7	-9.4	—	-9.2
	-8.6	-10.5	-9.8	—	—	-10.2	—	—	—

Table 1: **Similarity-constrained lead optimization (GenMol T4).** QuickVina2 docking score of the best feasible lead, lower is better, subject to QED ≥ 0.6 , SA ≤ 4 , and Tanimoto similarity $\geq \delta$ to the seed. Baseline values are from GenMol Table 4; bold marks the best entry in each cell. COMPOSE is feasible in 26/30 cells, matching GenMol, and improves the paired mean by 0.335 kcal/mol over the 23 cells where both are feasible (95% CI 0.019–0.651), using 500 docking-oracle evaluations per cell against 3,000 for the published GenMol result.

evaluations underlying the published GenMol result, a sixfold reduction. Since many individual docking differences are comparable to replicate variation, we use the paired aggregate as the primary comparison rather than interpreting small cell-level differences separately.

3.3 CAPABILITIES BEYOND FIXED-OBJECTIVE ENDPOINT OPTIMIZATION

Endpoint benchmarks score the molecule returned after optimization, but do not capture how decisions are made or revised along the trajectory. Because every COMPOSE state is a complete molecule, control can use downstream reachability, resume from a realized intermediate after an objective change, or restrict which molecular states may be visited.

Future reachability changes multi-step edit decisions. An edit that improves immediate similarity can still eliminate every successful continuation within the remaining budget. We test this on 65 held-out source–target pairs, each with a certified executable path to the target and sources disjoint from development. The local and future-aware policies share the same frozen R_θ , target, edit budget, and executable successors. They differ only in whether the next edit is selected by immediate target similarity or by the continuations available over the remaining budget. Local selection reaches 26 of 65 targets, whereas future-aware control reaches 40, rescuing 14 of the 39 local failures without losing a local success (Table 3). In one deterministic rescue, the locally preferred edit has higher immediate similarity but removes the successful continuation, while a slightly less similar edit preserves a path to the target. Exhaustive lookahead is not required: using h_ϕ to prioritize a single successor preserves the 40/65 recovery rate while evaluating 34.6% of the continuations required by all-candidate lookahead. The full prioritization comparison is reported in Appendix K.2.

Realized molecular states can be retargeted after an objective change. Starting from x_0 , we execute three edits under an initial objective and then switch objectives at the realized state x_3 . We compare continuing from x_3 with restarting from x_0 , holding the new objective and remaining edit budget fixed. Across 40 held-out molecules, continuing from x_3 improves post-switch utility in both directions: 0.302 [0.186, 0.413] for the potency-first objective and 0.176 [0.069, 0.283] for the developability-first objective (Table 3). The continued trajectories also move toward the new objective rather than preserving the preference under which x_3 was reached. Retargeting requires no update to R_θ ; control resumes from the realized molecular state under the new objective.

Pathwise constraints act on the molecular trajectory. Some molecular requirements must hold at every intermediate rather than only for the returned endpoint. COMPOSE enforces such statewise constraints by removing violating successors before the reference kernel is normalized. We compare this support restriction with endpoint-only filtering under matched sources, objectives, and edit budgets. On the developmental panel, 16 of 24 endpoint-feasible trajectories contained at least one intermediate constraint violation, showing that endpoint feasibility can hide inadmissible paths. Support-restricted control excludes such intermediates by construction. We report the matched effect on optimization performance and the full protocol in Appendix K.7.

Method	Trajectories/source	Success $\uparrow$	Diversity $\uparrow$
GrIDDD	20	45.1%	0.283
COMPOSE	8	55.8%	0.4999

Table 2: **Similarity-constrained QED editing on ZINC-250k.** A source is solved if at least one returned molecule reaches QED ≥ 0.9 while retaining Morgan-fingerprint Tanimoto similarity ≥ 0.4 to the source. GrIDDD is trained directly on ZINC-250k; COMPOSE transfers its GuacaMol-trained R_θ without retraining, and solves more sources using fewer than half as many candidate trajectories per source (95% CI [52.3, 59.2]). Diversity is the mean within-source pairwise Tanimoto distance over distinct non-source candidates, averaged over solved sources returning at least two; GrIDDD does not fully specify its diversity evaluator, so the two diversity entries are reported as published rather than as an exactly matched comparison (Appendix L.1).

Trajectory-level capability	Matched comparison	Main finding
Future-aware control	remaining-budget vs. local selection	40/65 vs. 26/65 targets reached
Dynamic retargeting	continue from x_3 vs. restart from x_0	improves post-switch utility in both objective switches
Pathwise constraints	support restriction vs. endpoint-only filtering	endpoint-only filtering permits hidden intermediate violations; support restriction prevents them by construction

Table 3: **Trajectory-level capabilities of the frozen molecular process.** Each comparison changes how the trajectory is controlled while keeping the reference process R_θ fixed. Confidence intervals, prioritization results, and full protocols are reported in Appendix K.

4 DISCUSSION

COMPOSE is built around a reference process that is learned independently of any downstream objective. In our experiments, the same GuacaMol-trained R_θ is reused for variable-size editing on ZINC-250k and for protein-specific lead optimization, with only the controller changing between tasks. Representing the intermediate states explicitly as molecules also makes it possible to look ahead before committing an edit, continue from a molecule already reached when the objective changes, and restrict which molecular states can be visited along a trajectory.

The current formulation still has important limits. Exact finite-horizon control requires exact backward values, which are only tractable on small enumerable state spaces. At molecular scale we instead rely on the learned approximation h_ϕ , so the exact terminal-law guarantee no longer applies. COMPOSE is also limited by the rewrite vocabulary used to define its support. Chemistry that cannot be expressed by those rewrites cannot be recovered through control. Finally, the optimization experiments inherit the limitations of the property models and docking scores used as objectives. Extending the rewrite process to broader chemistry and testing it against experimentally measured objectives are natural next steps.

REFERENCES

- Andrew Campbell, Jason Yim, Regina Barzilay, Tom Rainforth, and Tommi Jaakkola. Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design. In *International Conference on Machine Learning (ICML)*, 2024.
- Tong Chen, Yinuo Zhang, and Pranam Chatterjee. AReURDi: Annealed rectified updates for refining discrete flows with multi-objective guidance. *arXiv preprint arXiv:2510.00352*, 2025.

-
- Tong Chen, Maximilian Holsman, Lin Zhao, Yinuo Zhang, and Pranam Chatterjee. pCoMole: Pareto-constrained molecule editing with discrete flows. OpenReview, 2026. OpenReview submission <https://openreview.net/forum?id=LpttbUObQL>.
- Tianfan Fu, Cao Xiao, Xinhao Li, Lucas M. Glass, and Jimeng Sun. MIMOSA: Multi-constraint molecule sampling for molecule optimization. In *AAAI Conference on Artificial Intelligence*, 2021.
- Shrey Goel and Pranam Chatterjee. Minimal-action discrete Schrödinger bridge matching for peptide sequence design. *arXiv preprint arXiv:2601.22408*, 2026.
- Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams, and Alán Aspuru-Guzik. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2):268–276, 2018.
- Marton Havasi, Brian Karrer, Itai Gat, and Ricky T. Q. Chen. Edit flows: Flow matching with edit operations. *arXiv preprint arXiv:2506.09018*, 2025.
- Peter Holderrieth, Marton Havasi, Jason Yim, Neta Shaul, Itai Gat, Tommi Jaakkola, Brian Karrer, Ricky T. Q. Chen, and Yaron Lipman. Generator matching: Generative modeling with arbitrary markov processes. In *International Conference on Learning Representations*, 2025.
- Cristian Jensen, Luca Schaufelberger, Riccardo De Santi, Kjell Jorner, and Andreas Krause. Value matching: Scalable and gradient-free reward-guided flow adaptation. In *International Conference on Learning Representations (ICLR)*, 2026. OpenReview <https://openreview.net/forum?id=7iXt44Actj>.
- Jan H. Jensen. A graph-based genetic algorithm and generative model/monte carlo tree search for the exploration of chemical space. *Chemical Science*, 10(12):3567–3572, 2019.
- Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Junction tree variational autoencoder for molecular graph generation. In *International Conference on Machine Learning (ICML)*, 2018.
- Wengong Jin, Kevin Yang, Regina Barzilay, and Tommi Jaakkola. Learning multimodal graph-to-graph translation for molecular optimization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1812.01070.
- Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Hierarchical generation of molecular graphs using structural motifs. In *International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 4839–4848, 2020.
- Seul Lee, Karsten Kreis, Srimukh Prasad Veccham, Meng Liu, Danny Reidenbach, Yuxing Peng, Saeed Gopal Paliwal, Weili Nie, and Arash Vahdat. GenMol: A drug discovery generalist with discrete diffusion. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 33205–33226, 2025. URL <https://proceedings.mlr.press/v267/lee25o.html>.
- Matteo Ninniri, Marco Podda, and Davide Bacciu. Graph diffusion that can insert and delete. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.15725.
- Yifan Niu, Ziqi Gao, Tingyang Xu, Yang Liu, Yatao Bian, Yu Rong, Junzhou Huang, and Jia Li. InversionGNN: A dual path network for multi-property molecular optimization. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2503.01488.
- Sophia Tang, Yinuo Zhang, and Pranam Chatterjee. PepTune: De novo generation of therapeutic peptides with multi-objective-guided discrete diffusion. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2412.17780.
- Clément Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. Digress: Discrete denoising diffusion for graph generation. In *International Conference on Learning Representations*, 2023.

-
- Yutong Xie, Chence Shi, Hao Zhou, Yuwei Yang, Weinan Zhang, Yong Yu, and Lei Li. MARS: Markov molecular sampling for multi-objective drug discovery. In *International Conference on Learning Representations (ICLR)*, 2021.
- Seokwon Yoon, Youngbin Choi, Seunghyuk Cho, Seungbeom Lee, MoonJeong Park, and Dongwoo Kim. Routing by reaching: Composition of pre-trained GFlowNets for multi-objective generation. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.21565.
- Zhenpeng Zhou, Steven Kearnes, Li Li, Richard N. Zare, and Patrick Riley. Optimization of molecules via deep reinforcement learning. *Scientific Reports*, 9(1):10752, 2019.

APPENDIX

A RELATED WORK

Molecular generation spans latent-variable, graph-to-graph, reinforcement-learning, diffusion, and flow-based approaches, while molecular optimization has also been formulated directly through graph edits, fragment proposals, evolutionary operators, and MCMC transitions (Gómez-Bombarelli et al., 2018; Jin et al., 2018; 2019; Zhou et al., 2019; Jensen, 2019; Xie et al., 2021; Fu et al., 2021; Vignac et al., 2023; Campbell et al., 2024). Recent work has begun to relax the fixed-size assumption of graph generation: GrIDDD introduces node insertion and deletion into discrete graph diffusion, allowing molecular graphs to grow and shrink during denoising (Ninniri et al., 2025). COMPOSE instead learns a goal-independent stochastic law over an explicit, state-dependent molecular rewrite space, making the transition process itself the object of subsequent control.

A related line of work treats generation and adaptation directly through stochastic processes. Generator matching provides a general framework for learning Markov generators from tractable conditional processes, while Edit Flows develops insertion, deletion, and substitution dynamics for variable-length discrete generation (Holderrieth et al., 2025; Havasi et al., 2025). MadSBM is a close mathematical relative, formulating peptide generation as a controlled Markov process on an amino-acid edit graph, although its learned control constructs the generative transport rather than adapting an already learned molecular process (Goel and Chatterjee, 2026). Value Matching steers a frozen flow model through a learned value function, and pCoMole studies Pareto-constrained molecular editing with discrete flows (Jensen et al., 2026; Chen et al., 2026). Future-aware generative control is therefore not unique to COMPOSE: value-guided flow adaptation and pCoMole likewise use learned or rollout-estimated future value. Our distinction is the substrate on which that control operates. It is a learned canonical transition law over executable molecular graphs, which is what allows realized molecular states to be reused, retargeted, and constrained.

Multi-objective molecular design has been addressed through surrogate-based optimization, evolutionary search, and inference-time guidance of generative models. InversionGNN combines multi-property prediction with gradient-based Pareto search in discrete molecular space (Niu et al., 2025), while PepTune and AReURDi use discrete generative models with multi-objective tree search or Pareto-oriented refinement (Tang et al., 2025; Chen et al., 2025). Routing by Reaching similarly composes pretrained GFlowNets for multi-objective generation at inference time (Yoon et al., 2026). COMPOSE instead places multi-objective control on a reusable molecular transition process, so that future reachability and already-reached molecular states can participate directly in Pareto exploration.

B PRELIMINARIES

B.1 MARKOV CHAINS

Let $\mathcal{X}$ be a discrete state space and $P_0, \dots, P_{K-1}$ a sequence of transition kernels defining a possibly time-inhomogeneous Markov chain over K steps. Starting from $X_0 = x_0$, the probability of a trajectory $x_{1:K}$ is

$$P(X_{1:K} = x_{1:K} \mid X_0 = x_0) = \prod_{k=0}^{K-1} P_k(x_{k+1} \mid x_k). \quad (7)$$

Thus each transition depends only on the current state and the step of the process, while the sequence of kernels determines a distribution over complete trajectories. In COMPOSE, fixed-budget molecular editing takes this form: each state is a complete molecular graph and each committed transition corresponds to one molecular edit.

B.2 FINITE-HORIZON DOOB CONTROL

We also recall finite-horizon control of a reference Markov chain. Let $g_z(x) \geq 0$ assign terminal desirability under goal z . Defining the backward value $h_K(x, z) = g_z(x)$ and

$$h_k(x, z) = \sum_y P_k(y \mid x) h_{k+1}(y, z), \quad P_k^z(y \mid x) = P_k(y \mid x) \frac{h_{k+1}(y, z)}{h_k(x, z)}, \quad (8)$$

gives the finite-horizon Doob h -transform wherever $h_k(x, z) > 0$. Writing P^z for the law of the chain run under these controlled kernels, the resulting terminal distribution is

$$P^z(X_K = y \mid X_0 = x_0) = \frac{P(X_K = y \mid X_0 = x_0) g_z(y)}{\mathbb{E}_P[g_z(X_K) \mid X_0 = x_0]}. \quad (9)$$

The transform therefore reweights only transitions already present in the reference chain and introduces no new support. With exact backward values, it realizes the reference terminal distribution tilted by g_z . In COMPOSE, this construction provides the control law for molecular editing, with an amortized future-value model replacing exact backward computation in large molecular spaces.

COMPOSE instantiates these generative and control primitives on a state-dependent, trans-dimensional molecular rewrite space.

C THE EXECUTABLE MOLECULAR PROCESS IN FULL

We develop COMPOSE as a reusable stochastic process over molecular edits and a finite-horizon controller over that process. COMPOSE realizes generative stochastic rewriting by repeatedly applying executable molecular edits under a learned, goal-independent transition law. An exact rewrite system first determines which transitions between complete molecular graphs are executable. We then learn a goal-independent reference law over this state-dependent support and push it forward to a canonical molecular successor kernel R_θ . Finally, we control the frozen successor process through a budget-conditioned h -transform, using a learned future-value model h_ϕ to approximate the required backward values at molecular scale.

C.1 EXECUTABLE MOLECULAR TRANSITIONS

COMPOSE represents every generative state as a complete molecular graph. Let $n_{\max}$ denote the maximum number of heavy atoms admitted by the declared molecular state space. For $n \in \{1, \dots, n_{\max}\}$, let $\mathcal{X}_n$ be the set of connected molecular graphs with n heavy atoms whose node and edge attributes lie in the declared vocabularies of elements, bond orders, formal charges, and implicit hydrogens, and in which every atom satisfies its declared valence constraints. The molecular state space is

$$\mathcal{X} = \bigsqcup_{n=1}^{n_{\max}} \mathcal{X}_n. \quad (10)$$

Thus each state $x \in \mathcal{X}$ is a complete molecule rather than a masked, corrupted, or partially specified representation.

Molecular transitions are generated by typed graph rewrites. An edit mark a specifies a rewrite type together with its molecular match and typed payload, and $T(x, a)$ denotes the graph obtained by applying that rewrite to x . The full atom-, bond-, and ring-level edit language is specified in Appendix F.2. From a state x , the executor exposes only the finite, state-dependent set

$$\mathcal{A}(x) = \{a : a \text{ matches } x \text{ and } T(x, a) \text{ passes all executor guards}\}. \quad (11)$$

Execution follows a propose–validate–commit rule: a successor is admitted only if its attributes remain in the declared vocabularies, affected valences are satisfied, the graph remains connected, and its heavy-atom count lies in $\{1, \dots, n_{\max}\}$. An edit that fails any guard contributes no transition to $\mathcal{A}(x)$. Inadmissible molecular states are therefore absent from the stochastic support rather than discouraged through a learned penalty.

The decomposition of $\mathcal{X}$ by heavy-atom count makes the process trans-dimensional. Each admitted edit changes heavy-atom count by at most one, yielding *birth*, *death*, or *same-cardinality* transitions between $\mathcal{X}_n$ and $\mathcal{X}_{n+1}$, $\mathcal{X}_{n-1}$, or $\mathcal{X}_n$, respectively. Birth and death edits create or remove typed graph entities that participate in valence, connectivity, and ring perception rather than merely activating or deactivating padded coordinates. A trajectory may therefore grow, shrink, and restructure a molecular graph within a common state space.

Because membership in $\mathcal{A}(x)$ already encodes the guards defining $\mathcal{X}$, we have $T(x, a) \in \mathcal{X}$ for every $x \in \mathcal{X}$ and $a \in \mathcal{A}(x)$. Consequently every state reached by a stochastic process supported on $\{\mathcal{A}(x)\}_{x \in \mathcal{X}}$ remains in the declared molecular state space; the argument is given in Appendix G.

The legal edit sets $\mathcal{A}(x)$ therefore determine which molecular transitions are possible, but not how probability should be distributed among them. We next learn a goal-independent stochastic law over this executable support.

C.2 LEARNED MOLECULAR REFERENCE PROCESS

Having defined which molecular rewrites are executable, we next learn how probability is distributed over them. For every state x with $\mathcal{A}(x) \neq \emptyset$, the reference model defines a normalized conditional law over legal edit marks,

$$p_\theta(a \mid x, t), \quad \sum_{a \in \mathcal{A}(x)} p_\theta(a \mid x, t) = 1, \quad (12)$$

where t is a progress variable. Writing an edit as $a = (k, o, \eta)$, with family k , matched molecular operands o , and typed payload η , we parameterize

$$p_\theta(a \mid x, t) = p_\theta(k \mid x, t) p_\theta(o, \eta \mid k, x, t). \quad (13)$$

A shared molecular encoder represents the current graph, while family-specific scorers normalize only over candidates admitted by the executor. The reference model receives no downstream objective, remaining budget, or task-specific constraint, and the original source molecule is not supplied as an additional conditioning variable. The source determines the initial state; task information enters later through support restrictions and control. Thus the learned reference law represents molecular *plausibility* independently of molecular *purpose*.

Supervision is obtained by compiling molecular endpoint pairs into executable edit programs. Every stored teacher transition is replayed through the same executor used at inference, so the model is trained only on transitions available to the deployed process. The compiler may use both endpoints to construct an edit program, but neither the target endpoint nor the program is available to the reference model at inference. Because multiple legal marks can execute to the same molecular graph, the learning objective is defined on the resulting molecular successor rather than on a particular syntactic realization. Let $R_{\theta,t}(y \mid x)$ denote the induced law over distinct committed molecular successors, formalized in Appendix C.3. For productive teacher transitions $x_i \rightarrow y_i^*$, we minimize

$$\mathcal{L}_{\text{ref}}(\theta) = - \frac{\sum_i w_i \log R_{\theta,t_i}(y_i^* \mid x_i)}{\sum_i w_i}, \quad (14)$$

where w_i is the registered weight under the declared editing measure. Terminal rows do not enter this objective. Equation (14) therefore learns which committed molecular successor follows a state under that measure; it does not calibrate continuous-time holding rates.

This objective is related to generator matching but is not the generator-matching population objective. Writing the productive rate as $Q^+(y \mid x, t) = \Lambda(x, t)P(y \mid x, t)$, the corresponding generator-matching divergence separates pointwise into a productive-hazard term and a conditional-successor term proportional to $\Lambda^* \text{KL}(P^* \parallel R_\theta)$. COMPOSE optimizes the successor component only: the hazard term is not optimized, and Equation (14) is averaged under the declared family-weighted editing measure rather than the state–time marginal of the conditional-process mixture (Holderrieth et al., 2025). Appendix G.2 gives the full decomposition and the precise measure-level distinction.

The learned mark law is not yet the molecular kernel used for control: distinct executable marks can yield the same molecular state. We therefore next push p_θ through the executor and aggregate probability by canonical molecular outcome.

C.3 CANONICAL MOLECULAR SUCCESSORS

We now push the learned edit law through the executor to obtain a stochastic process over distinct molecular states. Multiple legal marks can produce the same molecular graph because of graph symmetries, equivalent matches, or alternative internal rewrite descriptions. For a canonical successor y , define its successor fiber

$$\mathcal{G}_y(x) = \{a \in \mathcal{A}(x) : \text{T}(x, a) \simeq y\}, \quad (15)$$

where $\simeq$ denotes equivalence under the registered molecular canonicalization. The induced probability of y is

$$\bar{p}_{\theta,t}(y \mid x) = \sum_{a \in \mathcal{G}_y(x)} p_\theta(a \mid x, t). \quad (16)$$

Thus probability assigned to a molecular transition is the aggregate mass of every executable mark that realizes it. For example, deleting any of the three symmetry-equivalent terminal methyl carbons in 2-methylpropane, CC(C)C, produces propane; canonicalization exposes propane once with the combined probability of those marks rather than overweighting it because of representational multiplicity.

Fixed-budget editing counts committed changes of molecular state rather than internal mark draws. Let $\mathcal{S}(x)$ denote the distinct non-self canonical successors admitted by this convention. Conditioning the pushforward distribution on a committed state change defines the molecular reference kernel

$$R_{\theta,t}(y | x) = \frac{\mathbf{1}\{y \in \mathcal{S}(x)\} \bar{p}_{\theta,t}(y | x)}{\sum_{y' \in \mathcal{S}(x)} \bar{p}_{\theta,t}(y' | x)}. \quad (17)$$

If $\mathcal{S}(x) = \emptyset$, x is terminal for the committed molecular process. For $b > 0$, such a state has $h_b(x, z) = 0$ under the backward recursion of Appendix D.1; trajectories terminating before the declared horizon therefore carry zero mass in the exact- K -step controlled terminal law. Otherwise $R_{\theta,t}$ is a normalized law over distinct molecular successors, and iterating it defines the goal-independent reference process optimized by Equation (14). We suppress t and write $R_{\theta}(y | x)$ when the registered progress convention is fixed; canonical self-events and other non-committed outcomes are detailed in Appendix G.1.

Proposition 2 (Canonical normalization). *For every nonterminal state x , $\sum_{y \in \mathcal{S}(x)} R_{\theta,t}(y | x) = 1$. Moreover, any refinement or coarsening of the internal mark representation that preserves the aggregate probability mass of every canonical successor fiber leaves $R_{\theta,t}$ unchanged. Consequently, state-level controllers that reweight $R_{\theta,t}$ using quantities defined on canonical molecular successors are invariant to such representational refinements.*

This invariance relies on performing control after canonicalization: reweighting or truncating individual marks beforehand can reintroduce dependence on arbitrary edit multiplicities. With the goal-independent reference process now defined on complete molecular states, we next control this frozen kernel toward finite-horizon design objectives.

D FINITE-HORIZON CONTROL IN FULL

D.1 FINITE-HORIZON MOLECULAR CONTROL

Having defined the goal-independent molecular reference process R_{θ} , we now introduce task-specific control without modifying that process. Let z denote a design objective and $g_z : \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}$ its terminal desirability. For a remaining budget of b molecular transitions, define the backward values

$$h_0(x, z) = g_z(x), \quad h_b(x, z) = \sum_{y \in \mathcal{S}(x)} R_{\theta}(y | x) h_{b-1}(y, z) = \mathbb{E}_{R_{\theta}}[g_z(X_b) | X_0 = x]. \quad (18)$$

Thus $h_b(x, z)$ evaluates a molecule by the desirability reachable from it after b further transitions under the reference process. Unlike a score of the current molecule, it depends on the molecular futures that remain accessible through R_{θ} .

For $h_b(x, z) > 0$, the finite-horizon h -transform defines

$$R_{\theta}^{z,b}(y | x) = R_{\theta}(y | x) \frac{h_{b-1}(y, z)}{h_b(x, z)}. \quad (19)$$

The controlled kernel therefore reweights the same canonical molecular successors according to their remaining future value: a transition may be preferred despite little immediate improvement if it preserves access to desirable states later in the trajectory. If $h_b(x, z) = 0$, no state with positive terminal desirability is reachable within the remaining budget under the reference process, and we leave the controlled row undefined rather than introduce probability outside its support.

Theorem 3 (Finite-horizon molecular control). *Let R_{θ} define the reference path measure over K committed transitions from x_0 , with trajectories terminating before K assigned zero continuation mass, let g_z be nonnegative and finite-valued, and suppose $0 < h_K(x_0, z) < \infty$. Applying $R_{\theta}^{z,K-j}$ at step $j = 0, \dots, K-1$ defines normalized controlled kernels whose support is contained in that of R_{θ} , and whose terminal distribution satisfies*

$$\mathbb{P}^z(X_K = y | X_0 = x_0) = \mathbb{P}_{R_{\theta}}(X_K = y | X_0 = x_0) \frac{g_z(y)}{h_K(x_0, z)}. \quad (20)$$

Thus exact backward values realize the g_z -tilted terminal law without introducing transitions outside the executable reference support. The proof is given in Appendix G.3.

The guarantee is horizon-specific: before the terminal step, the controlled marginal is determined by the remaining backward value rather than directly by g_z , so early stopping does not in general realize the terminal tilt. Because every state of R_{θ} is a complete molecule and every transition an executable

rewrite, $h_b(x, z)$ has a direct interpretation as future desirability from a realized molecular intermediate through its remaining executable continuations. Exact backward evaluation is tractable only when the relevant state graph can be enumerated. Appendix D.2 therefore replaces h_b with a learned goal- and budget-conditioned approximation $h_\phi(x, z, b)$, retaining the same finite-horizon control interface at molecular scale.

D.2 LEARNING GOAL- AND BUDGET-CONDITIONED FUTURE VALUE

Exact evaluation of the backward values in Appendix D.1 requires propagating terminal desirability over the molecular states reachable within the remaining edit budget, which is infeasible in realistic molecular spaces. We therefore learn an amortized approximation h_ϕ while keeping the molecular reference process R_θ fixed. For objective z and b remaining transitions,

$$h_\phi(x, z, b) \approx h_b(x, z) = \mathbb{E}_{R_\theta}[g_z(X_b) \mid X_0 = x]. \quad (21)$$

The remaining budget is part of the prediction problem: the same molecule can have different future value under the same objective depending on how many executable transitions remain.

We construct supervision directly from the frozen reference process. From a molecular state x , M independent b -step continuations give the Monte Carlo target

$$\hat{h}_M(x, z, b) = \frac{1}{M} \sum_{m=1}^M g_z(X_b^{(m)}), \quad X_b^{(m)} \sim R_\theta^b(\cdot \mid x). \quad (22)$$

When g_z is an indicator, this target estimates finite-budget reachability; for a general nonnegative desirability, it estimates expected future value. Training and validation are partitioned by molecular source so that states descended from the same source do not cross splits.

The exact backward values also satisfy the Bellman recursion in Equation (18), which provides an additional source of supervision when adjacent-budget trajectory pairs are available. We use this consistency relation as an auxiliary training signal for the reachability-valued controller, while controllers trained on soft expected-desirability targets are fit directly to their Monte Carlo future-value estimates. In both cases, h_ϕ is trained to approximate the backward quantity defining the exact controller rather than an independent task-specific score. Architecture and training details for the respective future-value models are given in Appendices I.3 and I.4.

This distinction is important when terminal desirability is derived from a learned property model $\hat{F}_\psi$. The property model $\hat{F}_\psi(x)$ estimates properties of the current molecule, whereas $h_\phi(x, z, b)$ estimates the desirability reachable from that molecule after b further transitions of R_θ . Property learning therefore estimates where value may lie; future-value learning propagates that value through the molecular process.

Replacing the exact backward value in Equation (19) yields the scalable controlled kernel

$$P_\phi(y \mid x, z, b) = \frac{R_\theta(y \mid x) h_\phi(y, z, b-1)}{\sum_{y' \in \mathcal{S}(x)} R_\theta(y' \mid x) h_\phi(y', z, b-1)}, \quad y \in \mathcal{S}(x), \quad (23)$$

whenever the denominator is positive. If $h_\phi = h$, Equation (23) recovers the exact finite-horizon transform of Theorem 3. With learned h_ϕ , approximation error can change how probability is allocated among executable successors, but cannot introduce a transition outside the support of R_θ . Exact terminal reweighting remains a guarantee of the exact h -transform and is not claimed for the learned approximation.

D.3 SAMPLING THE CONTROLLED MOLECULAR PROCESS

At inference, COMPOSE applies the controlled kernel in Equation (23) recursively over realized molecular states. Given the current molecule x , objective z , and remaining budget b , the executor enumerates the canonical successor set $\mathcal{S}(x)$; R_θ assigns goal-independent reference probability to these successors, and h_ϕ reweights them by their predicted remaining future value. After sampling and committing a successor, the resulting molecule becomes the next state and the budget is decremented. Recomputing the controlled law after every committed edit therefore yields a closed-loop molecular trajectory whose decisions depend on the state actually reached rather than on a trajectory fixed in advance.

When $\mathcal{S}(x)$ can be evaluated directly, we sample from the normalized kernel in Equation (23). The same row can alternatively be sampled by rejection whenever h_ϕ admits a finite envelope: proposing

from $R_\theta(\cdot \mid x)$ and accepting in proportion to h_ϕ recovers the controlled distribution conditional on acceptance. For objectives whose desirable futures carry little reference mass, direct sampling can allocate insufficient computation to useful trajectories. We instead use a Feynman–Kac sequential Monte Carlo construction. Particles propagate under the frozen reference process R_θ , and a transition $x \rightarrow y$ with b remaining steps receives incremental potential

$$G_b(x, y) = \frac{h_\phi(y, z, b-1)}{h_\phi(x, z, b)}. \quad (24)$$

Particles are resampled when effective sample size falls below $N/2$. When h_ϕ equals the exact backward value, these potentials recover the finite-horizon Doob-controlled path law of Appendix D.1. With learned h_ϕ , the same value approximation acts as a twisting function for the reference particle system; the resulting finite-particle procedure is an approximation whose target and boundary conditions are detailed in Appendix J.

For objectives defined by a target region $\mathcal{B}_z$, we stop prospectively at the first realized hit,

$$\tau = \min(\{t \leq K : X_t \in \mathcal{B}_z\} \cup \{K\}). \quad (25)$$

A reached molecule is returned when the target region is entered; if no hit occurs, the trajectory continues to its declared budget. Intermediate states are not retrospectively collected as additional samples: one controlled trajectory produces one returned candidate. Objectives without a first-hit interpretation are sampled to the declared horizon.

D.4 PREFERENCE-CONDITIONED MULTI-OBJECTIVE CONTROL

The preceding construction applies to any nonnegative terminal desirability g_z . To control COMPOSE under competing molecular objectives, let $\hat{\mathbf{F}}_\psi(x) = (\hat{F}_{\psi,1}(x), \dots, \hat{F}_{\psi,m}(x)) \in [0, 1]^m$ denote frozen property scores oriented so that larger values are preferred, and let $\lambda \in \Delta^{m-1}$ specify a preference over objectives. We define the weighted Chebyshev shortfall

$$L_\lambda(x) = \max_{i=1, \dots, m} \lambda_i (1 - \hat{F}_{\psi,i}(x)). \quad (26)$$

Unlike an additive scalarization, the maximum is determined by the most limiting preference-weighted objective, preventing improvement in one property from arbitrarily compensating for poor performance on another.

We convert this shortfall into the nonnegative terminal desirability required by the finite-horizon controller,

$$g_\lambda(x) = \exp\left(-\frac{L_\lambda(x)}{\tau_\lambda}\right), \quad \tau_\lambda = \text{Std}_{x \sim \mathcal{D}_{\text{train}}}[L_\lambda(x)] > 0. \quad (27)$$

The scale τ_λ is fixed from the property-supervision set before molecular optimization and is not fitted to generated candidates. Numerical centering used in the implementation leaves normalized controlled transitions unchanged and is detailed in Appendix I.4.

Setting $z = \lambda$ in Appendix D.2 gives

$$h_\phi(x, \lambda, b) \approx \mathbb{E}_{R_\theta}[g_\lambda(X_b) \mid X_0 = x],$$

and Equation (23) then defines the corresponding preference-conditioned molecular process. Different values of λ therefore alter the desired tradeoff through the controller while leaving the learned molecular reference process R_θ fixed. Running the controller under multiple preferences yields preference-conditioned candidate sets, from which we report the nondominated subset; the scalarization itself carries no claim of Pareto optimality.

The same state-level formulation also permits objectives and admissibility requirements to change after generation has begun, as we describe next.

D.5 RETARGETING AND PATHWISE MOLECULAR CONSTRAINTS

Because control is conditioned on the current molecular state, COMPOSE can change either the design objective or the admissible transition set without retraining the molecular reference process. Suppose a trajectory controlled under objective z reaches the realized molecule x_τ with b transitions remaining, after which the objective changes to z' . Control resumes by evaluating the same kernel in Equation (23) from x_τ under the new goal context z' . The realized prefix $x_0 \rightarrow \dots \rightarrow x_\tau$ is retained, and no parameter of R_θ is updated.

Corollary 4 (Exact continuation after a goal change). *With exact backward values, applying Theorem 3 from (x_τ, b) with terminal desirability $g_{z'}$ realizes the $g_{z'}$ -tilted b -step terminal law of the reference process started at x_τ . This generally differs from specifying z' at the original source x_0 : continuation preserves the molecular state actually reached, whereas restarting discards it. With learned h_ϕ , the same operation gives the corresponding approximate continuation.*

Hard statewise requirements can instead be enforced through executable support. Let $C : \mathcal{X} \rightarrow \{0, 1\}$ be a molecular predicate with $C(x_0) = 1$, and define the restricted edit set

$$\mathcal{A}_C(x) = \{a \in \mathcal{A}(x) : C(T(x, a)) = 1\}. \quad (28)$$

Applying the canonicalization and normalization of Appendix C.3 to $\mathcal{A}_C(x)$ gives the constrained reference kernel R_θ^C . If $\mathcal{A}_C(x) = \emptyset$, the state is terminal under the constrained process; otherwise every committed successor satisfies C by construction. Finite-horizon control then proceeds with R_θ^C in place of R_θ . Approximation error in h_ϕ may therefore misallocate probability among admissible successors, but cannot violate a constraint enforced through the support itself.

This construction applies to state predicates whose admissibility can be determined from the proposed molecular successor; we refer to such constraints as *rule-closed*. Examples include protected scaffolds, forbidden substructures, and size or charge restrictions. Requirements that depend on accumulated history require an augmented state. For a sufficient statistic ξ_k , define

$$\tilde{X}_k = (X_k, \xi_k), \quad \xi_{k+1} = \Phi(\xi_k, X_{k+1}),$$

so that the augmented process remains Markov. Cumulative edit costs or other history-dependent requirements can then be represented explicitly rather than imposed through a non-Markov rejection rule.

Retargeting and pathwise restriction therefore modify different parts of the same frozen process: retargeting changes the control context, whereas hard constraints change executable support. In both cases, the parameters of the learned molecular reference process remain fixed.

E SUPPORTING MATERIAL

OVERVIEW OF APPENDIX

This appendix provides the formal, implementation, and experimental details supporting the main text. Appendix F specifies the executable molecular state space, rewrite semantics, and canonical molecular-successor representation. Appendix G gives the complete theoretical results underlying support preservation and finite-horizon control. Appendices H to J describe transition supervision, model training, and controlled sampling. Appendix K gives the experimental details and extended results for each experiment in the order they appear in the main text, Appendix L documents alignment with external benchmark protocols, and Appendix M records the software environment, randomization scheme, and compute accounting.

F MOLECULAR STATE SPACE AND EXECUTABLE EDITS

The support-level claims of COMPOSE are defined with respect to an explicit molecular state space rather than post-hoc sanitization. This section specifies that state space and the rewrite contracts that induce its transition graph. The *semantic state* is the active molecular graph on which connectivity, valence, ring perception, and canonical identity are evaluated, and we distinguish it from any padded tensor representation used by the implementation. We then define the current operator family, its exact legality checks, and the pushforward from executable rewrite marks to canonical molecular successors. These definitions determine what transitions are possible before either R_θ or a task-specific controller assigns probability to them.

F.1 MOLECULAR STATE SPACE

States carry a bounded number of heavy atoms over a declared set of neutral (element, valence) classes drawn from an organic element set; the declared bounds and vocabulary are fixed in the frozen process configuration and are identical across every experiment reported here. Formal charge is an input but not an output dimension, so charge is carried rather than designed; stereochemistry, isotopes and radicals are not represented.

Cardinality is a property of the active molecular graph rather than of a fixed tensor allocation. Writing $\mathcal{X}_n$ for the states with n heavy atoms, the state space is the disjoint union of the strata, and a birth or death moves a trajectory between strata by creating or destroying a typed graph entity that participates in valence, connectivity, and ring perception. This is not the activation or deactivation of a padded coordinate, and the distinction is what makes the process trans-dimensional in the sense used throughout the paper.

F.2 REWRITE OPERATORS

The main text depends only on the legal edit set (11) and never on which families populate it; this section records the families used, so that the edit set is reproducible. We use paper-facing operator names for executable rewrite semantics and model-family names when reporting learned-family statistics; Table 4 gives the fixed mapping between them.

Table 4: The rewrite families, by action on the strata. Each family is a typed graph rewrite with structural and chemical application conditions; the executor follows a propose–validate–commit discipline and exposes no transition whose successor fails a guard. Names here are the paper-facing operator names. The implementation additionally carries model-family identifiers, which appear in per-family results: `cycle_close` is recorded as `cycle_insert`, `cycle_open` as `cycle_attach`, and `ring_aromaticity_restate` as `ring_system_restate`. The remaining five names coincide in both registers. Per-family results in Appendix K.1 are reported under the model-family identifiers.

Family	Cardinality	Structural / chemical condition
<code>atom_insert</code>	birth $\mathcal{X}_n \rightarrow \mathcal{X}_{n+1}$	empty capacity; a null source or exactly one occupied anchor; allowed atom state; for a bonded birth, allowed bond order and feasible endpoint valence
<code>atom_delete</code>	death $\mathcal{X}_n \rightarrow \mathcal{X}_{n-1}$	removable atom with connected remainder; neighbour hydrogen compensation; successor sanitizes
<code>atom_restate</code>	same	occupied site; joint element, valence-class and implicit-hydrogen state allowed; successor differs from the source
<code>bond_reorder</code>	same	existing edge; joint bond-order and hydrogen update feasible
<code>bond_reroute</code>	same (reroute)	the cut edge is a bridge; the relocated fragment is a pendant subtree; the new endpoint lies outside it; cut and attach commit in one transaction so no disconnected state is ever exposed
<code>cycle_close</code>	same (cyclize)	absent edge between two occupied atoms with valence headroom; admitted insertion raises graph cycle rank by one
<code>cycle_open</code>	same (decyclize)	existing edge that is not a bridge; endpoint hydrogen compensation; successor sanitizes
<code>ring_aromaticity_restate</code>	same	a matched cyclic system; a coordinated set of bond-order changes validated by the executor as one transaction

The operators above are the complete executable support. Whole-ring growth and whole-ring deletion macros are not part of it: every ring in this work is built and modified through the primitive operators listed here.

`cycle_close` inserts one bond between two nonbonded atoms and, because every non-null state is connected, raises graph cycle rank by one; `cycle_open` deletes one non-bridge cycle edge and lowers it by one. At the untyped graph level every connected graph is a spanning tree plus its non-tree edges, so inserting those edges one at a time realizes the abstract cyclic topology while preserving connectivity. This establishes topological reachability at the untyped graph level, not the existence of a valence-valid molecular path within a finite edit budget. It does not prove that the molecular grammar admits a valence-valid path to every chemically labelled ring system, nor that such a path fits a finite edit budget; both are measured questions and neither is assumed anywhere in the main text.

F.3 COMPILED MULTI-EDIT TRANSFORMATIONS

The lead-optimization controller may select a multi-edit ring transformation, such as constructing or refining a ring system, rather than a single primitive edit. These transformations are compiled programs,

not new operators. A program of length L executes L primitive edits in sequence; at every step the successor is drawn from the same legal-successor enumeration $\mathcal{A}(\cdot)$ used everywhere else, applied through the same executor, and screened by the same guards. A program halts if a state admits no legal successor.

Two consequences matter for the claims in this paper. First, the executable support is unchanged: every endpoint a program can reach is reachable by the corresponding sequence of primitive rewrites, so compiling ring construction introduces no molecular successor that $\mathcal{A}(\cdot)$ does not already admit, and Appendix G therefore applies unchanged to the controlled process. Second, what a program changes is the granularity of control, not the support: purpose is evaluated at the program endpoint rather than after each edit, because a constructive ring program passes through intermediate states whose immediate objective value is worse than both its start and its endpoint. Scoring each edit separately prunes such a program before it completes, which is a property of the control schedule and not of the state space.

F.4 CANONICAL MOLECULAR SUCCESSORS

Distinct executable marks can produce the same molecule, so the law the controller acts on is the pushforward of the mark law through the executor, aggregated at canonical molecular identity as in (15)–(17). Aggregation is what makes every downstream quantity independent of how many ways the implementation can name a molecule; Appendix G.1 establishes the precise invariance this buys and its exact scope.

G THEORETICAL RESULTS

The deployed reference process. All fixed-budget experiments evaluate the reference model at the single progress value $t_0 = 0.5$. We therefore write

$$p_\theta(a \mid x) := p_\theta(a \mid x, t_0), \quad R_\theta(y \mid x) := R_{\theta, t_0}(y \mid x)$$

throughout, and the deployed molecular reference process is time-homogeneous. The results below are stated for that process.

Executable-support closure. By definition, an edit a belongs to $\mathcal{A}(x)$ only if its proposed successor passes the attribute, valence, connectivity and heavy-atom-count conditions that define $\mathcal{X}$. Hence

$$x \in \mathcal{X}, \quad a \in \mathcal{A}(x) \implies T(x, a) \in \mathcal{X}.$$

Induction over committed transitions shows that any process whose transition support is contained in the executable edit set remains in $\mathcal{X}$. This applies to the reference process and to any controlled or support-restricted process derived from it, since every supported successor is the canonical image of an edit admitted by $\mathcal{A}(x)$.

G.1 CANONICAL SUCCESSOR REPRESENTATION

Definition 5 (Successor-mass-preserving re-encoding). Consider two executable mark representations $(\mathcal{A}, p_\theta, T)$ and $(\mathcal{A}', p'_\theta, T')$ of the same molecular state space. They are *successor-mass preserving* if they induce the same committed canonical successor set $\mathcal{S}(x)$ and, for every x and every $y \in \mathcal{S}(x)$,

$$\sum_{\substack{a \in \mathcal{A}(x) \\ T(x, a) \simeq y}} p_\theta(a \mid x) = \sum_{\substack{a' \in \mathcal{A}'(x) \\ T'(x, a') \simeq y}} p'_\theta(a' \mid x).$$

Proposition 6 (Invariance under successor-mass-preserving re-encoding). *Under a successor-mass-preserving re-encoding, the canonical pushforward distribution and the committed molecular kernel $R_\theta(y \mid x)$ are unchanged. Consequently any state-level controller*

$$K_\Psi(y \mid x) = \frac{R_\theta(y \mid x) \Psi(x, y)}{\sum_{y'} R_\theta(y' \mid x) \Psi(x, y')}, \quad \Psi \geq 0,$$

is unchanged wherever the denominator is positive.

Proof. The assumption gives equal probability mass for every canonical successor y . Summing those equal masses over the common committed successor set gives the same normalization constant, so $R_\theta(y \mid x)$ is identical under both representations. A state-level controller has the same numerator and denominator because both R_θ and Ψ are functions of canonical molecular states. For a predicate C defined on canonical states, all marks in a successor fiber are retained or removed together, so the restricted successor masses and their normalization are also identical under both encodings. $\square$

Why control follows canonicalization. Suppose a canonical successor has total mark mass p . A nonlinear mark-level reweighting $u \mapsto u^\beta$ assigns it mass p^β when it is represented by one mark, but $m(p/m)^\beta$ when it is represented by m equal aliases. These agree only when $\beta = 1$. Nonlinear guidance applied at mark level can therefore depend on arbitrary representational multiplicity, while successor-level control cannot.

G.2 RELATION TO GENERATOR MATCHING

The reference model is related to generator matching through the conditional-successor component of the productive generator. This is a statement about the training objective, so we state it pointwise at an arbitrary progress value: for any t , write $Q^+(y \mid x, t) = \Lambda(x, t) P(y \mid x, t)$, separating a total productive exit rate from a normalized law over canonical molecular successors. Deployment then fixes $t = t_0$ as above. For the Bregman divergence generated by $\varphi(u) = u \log u - u$, direct expansion gives

$$D_\varphi(Q^{+\star}, Q_\theta^+) = \Lambda^\star \text{KL}(P^\star \parallel R_{\theta, t}) + \Lambda^\star \log \frac{\Lambda^\star}{\Lambda_\theta} - \Lambda^\star + \Lambda_\theta. \quad (29)$$

The first term depends on the normalized successor law, and the remaining terms depend on the productive exit rate.

For a stored COMPOSE teacher row $(x_i, t_i, y_i^\star)$, the compiled supervision records a productive successor identity but no holding time or rate magnitude. Under the unit-intensity representation used for this comparison, $\Lambda_i^\star = 1$ and $P_i^\star = \delta_{y_i^\star}$, so the conditional-successor block of that row is

$$\Lambda_i^\star \text{KL}(P_i^\star \parallel R_{\theta, t_i}) = -\log R_{\theta, t_i}(y_i^\star \mid x_i).$$

The weighted empirical average of these rowwise terms is therefore exactly the productive-successor objective (14), up to factors independent of θ .

COMPOSE optimizes an empirical version of the conditional-successor block, and differs from the generator-matching population objective in exactly two respects. First, the hazard block is never optimized. The compiled supervision carries no rate magnitudes: every stored transition is registered with unit teacher rate, which enters training only as a nonterminality indicator, so no target for Λ_θ exists in the corpus. Under that unit-rate convention $\Lambda^\star$ is constant and the weight it contributes to the first term is absorbed into a positive constant independent of θ . The reported results never read an exit-rate magnitude; fixed-budget editing consumes only the embedded successor kernel, on which a constant rescaling of Q^+ has no effect.

Second, the expectation is taken under a different measure. Generator matching integrates the per-state divergence against the state–time marginal of the conditional-process mixture. COMPOSE instead draws training rows under a declared editing measure that is weighted across operator families by design, so that a family’s exposure reflects the reference law the model is meant to estimate rather than the incidental composition of the compiled corpus; some strata are assigned zero draw. No importance correction is applied, and on zero-draw strata none could be. Consequently (29) holds pointwise in (x, t) , and (14) is the conditional-successor block of that decomposition, but the training functional is not the generator-matching population objective. We therefore describe the relationship as a decomposition that identifies which factor R_θ estimates, not as an objective identity.

G.3 FINITE-HORIZON CONTROL

Terminal states. To make early termination explicit, adjoin a cemetery state ∂ . If a molecular state has no committed successor, the reference process moves to ∂ with probability one, and ∂ is absorbing. We set $g_z(\partial) = 0$. This is equivalent to assigning zero continuation mass to trajectories that terminate before the declared horizon, and it makes R_θ and h_b defined everywhere.

Theorem 7 (Finite-horizon molecular control; restatement of Theorem 3). *Let R_θ be the reference kernel above and let $g_z : \mathcal{X} \cup \{\partial\} \rightarrow \mathbb{R}_{\geq 0}$ be finite-valued with $g_z(\partial) = 0$. Define*

$$h_0(x, z) = g_z(x), \quad h_b(x, z) = \sum_y R_\theta(y \mid x) h_{b-1}(y, z),$$

and suppose $0 < h_K(x_0, z) < \infty$. Wherever $h_b(x, z) > 0$, define

$$P^{z, b}(y \mid x) = R_\theta(y \mid x) \frac{h_{b-1}(y, z)}{h_b(x, z)}.$$

Then each controlled row is normalized and has support contained in that of R_θ , and applying these kernels with remaining budgets $K, K-1, \dots, 1$ gives

$$\Pr_{P^z}(X_K = y \mid X_0 = x_0) = \Pr_{R_\theta}(X_K = y \mid X_0 = x_0) \frac{g_z(y)}{h_K(x_0, z)}.$$

Proof. For any state with $h_b(x, z) > 0$,

$$\sum_y P^{z,b}(y | x) = \frac{\sum_y R_\theta(y | x) h_{b-1}(y, z)}{h_b(x, z)} = 1$$

by the backward recursion. If $R_\theta(y | x) = 0$ then $P^{z,b}(y | x) = 0$, so control introduces no reference-forbidden transition. Moreover positive controlled probability implies $h_{b-1}(y, z) > 0$, so every nonterminal state reached by the controlled process has a well-defined controlled row at the next step. For any K -step path $x_{0:K}$ of positive controlled probability,

$$\begin{aligned} \Pr_{P^z}(x_{1:K} | x_0) &= \prod_{j=0}^{K-1} R_\theta(x_{j+1} | x_j) \frac{h_{K-j-1}(x_{j+1}, z)}{h_{K-j}(x_j, z)} \\ &= \Pr_{R_\theta}(x_{1:K} | x_0) \frac{h_0(x_K, z)}{h_K(x_0, z)} = \Pr_{R_\theta}(x_{1:K} | x_0) \frac{g_z(x_K)}{h_K(x_0, z)}, \end{aligned}$$

because the interior h -factors telescope and $h_0(x_K, z) = g_z(x_K)$. Summing over all reference paths ending at y gives the stated terminal law. $\square$

Remark 8 (Horizon specificity). For $j < K$ the same calculation gives

$$\Pr_{P^z}(X_j = x | X_0 = x_0) = \Pr_{R_\theta}(X_j = x | X_0 = x_0) \frac{h_{K-j}(x, z)}{h_K(x_0, z)}.$$

Intermediate states are therefore tilted by remaining future value rather than directly by g_z , and stopping before the declared horizon does not in general realize the terminal g_z -tilt.

Learned future value. At molecular scale we replace h_b with a learned approximation $h_\phi(x, z, b)$. All deployed future-value heads use a sigmoid output, so $0 < h_\phi(x, z, b) < 1$. Consequently

$$P_\phi(y | x, z, b) = \frac{R_\theta(y | x) h_\phi(y, z, b-1)}{\sum_{y'} R_\theta(y' | x) h_\phi(y', z, b-1)}$$

is a well-defined probability distribution wherever $R_\theta(\cdot | x)$ is defined. Whenever the learned sigmoid head is used, its strict positivity preserves the reference support. At the terminal step the exact boundary $h_0 = g_z$ may remove zero-desirability successors, for example when g_z is a region indicator. In all cases the controlled support is contained in that of R_θ and no new transition can be introduced. The exact terminal-law result above does not carry over unless $h_\phi = h$.

G.4 RETARGETING AND PATHWISE SUPPORT

Corollary 9 (Continuation after a goal change). *Suppose a realized trajectory reaches x_τ with b transitions remaining, after which the objective changes from z to z' . If $0 < h_b(x_\tau, z') < \infty$, exact finite-horizon control from x_τ under terminal desirability $g_{z'}$ gives*

$$\Pr_{P^{z'}}(X_b = y | X_0 = x_\tau) = \Pr_{R_\theta}(X_b = y | X_0 = x_\tau) \frac{g_{z'}(y)}{h_b(x_\tau, z')}.$$

Retargeting therefore solves the remaining finite-horizon control problem from the molecular state actually reached, which differs in general from restarting under z' from x_0 .

Proof. Apply Theorem 7 with initial state x_τ , horizon b , and terminal desirability $g_{z'}$. $\square$

This is a correctness statement about continuation. It does not imply that continuing from x_τ must outperform restarting from x_0 ; that question is evaluated empirically in Section 3.3.

Proposition 10 (Pathwise state constraints). *Let $C : \mathcal{X} \rightarrow \{0, 1\}$ be a predicate on canonical molecular states with $C(x_0) = 1$, and define $\mathcal{A}_C(x) = \{a \in \mathcal{A}(x) : C(T(x, a)) = 1\}$. Let R_θ^C be the canonical successor kernel obtained from $\mathcal{A}_C(x)$ by the same pushforward and normalization used to construct R_θ . If $\mathcal{A}_C(x) = \emptyset$ the state is terminal under the constrained process. Otherwise every committed state under R_θ^C satisfies C , and the same holds for any controlled kernel whose support is contained in that of R_θ^C .*

Proof. The initial state satisfies C . Suppose X_k satisfies C . Every mark in $\mathcal{A}_C(X_k)$ has a successor satisfying C by definition, so every state that can be committed at step $k+1$ also satisfies C . Induction proves the claim. A controller supported on R_θ^C can only remove or reweight admissible successors and therefore preserves the invariant. $\square$

This construction applies directly when admissibility can be decided from the proposed successor. A history-dependent requirement instead needs a Markov state augmentation: if $\xi_{k+1} = \Phi(\xi_k, X_{k+1})$ is a sufficient running statistic, the same support-restriction argument applies on the augmented state (X_k, ξ_k) .

H DATA AND TRANSITION SUPERVISION

The supervision used to fit R_θ varies across the executable edit space. We therefore specify the molecular source pool, the construction of executable transition supervision, and the associated data splits and provenance, since these determine which reference-law results are supported by data-backed transitions and which rely on synthetic perturbations. This variation bounds how the strongest per-family results in Appendix K.1 may be read.

H.1 MOLECULAR DATA AND PREPROCESSING

The editing corpus was derived from a fixed 500,000-molecule GuacaMol source pool by compiling executable local transitions, real-endpoint analogue relations, and reversible synthetic walks. COMPOSE was not trained as an unconditional distribution model over that pool: the learned object is $R_\theta(y | x)$, a transition law over source-successor edits, not a density $p_\theta(x)$ over molecules.

The source pool is identified by the recorded dataset namespace and its SHA-256, and independently corroborated by the molecular statistics of the frozen training sources. Measured on 4,000 randomly sampled training sources, the element distribution is exactly the declared organic vocabulary, with carbon at 0.748, nitrogen and oxygen at 0.105 each, sulfur at 0.015, fluorine at 0.012, chlorine at 0.009, bromine at 0.003, iodine and phosphorus at 0.002, and boron at 0.001. The median heavy-atom count is 26, median molecular weight 371.6 and ninetieth percentile 499.3, median ring count 3, median aromatic-ring count 2, and only 1.6% acyclic. Because the training receipts do not bind the original raw asset end to end, we treat this provenance link as strongly supported rather than cryptographically complete; the frozen compiled corpus itself is exactly identified by its dataset, library, packed-store and reserve digests. The benchmark sources of Appendix L.1 are drawn from a different pool.

H.2 TRANSITION CONSTRUCTION

Transitions enter the corpus through several lanes, which are near family-pure with lane-homogeneous chunks. The frozen structural gate records 1,803,032 training transitions passing with zero violations. A metadata scan of every training chunk attributes `cycle_insert` (122,183 transitions across 102 chunks), `cycle_attach` (84,940 across 102) and `ring_system_restate` (16,380 across 29) entirely to the reversible synthetic-walk lane, with zero transitions in any of the four data-backed lanes; in the pinned distribution-matched training subset that lane contributes 23,100 of 145,756 transitions, or 15.8%, against 84.2% data-backed. By contrast `bond_reroute` and `atom_restate` each appear in two lanes and therefore do have a real-chemistry source. The analogue lane draws on a frozen molecular-matched-pair pool of 344,153 records bound by a recorded contract hash, whose own upstream molecular source is not fully documented.

Accordingly, results on the ring and cycle families support recovery of legal ring edits under synthetic perturbation, but do not establish data-backed ring editing or scaffold hopping. Establishing the latter would require supervision from observed ring transformations, which is absent from the current corpus.

H.3 SUPERVISION PROVENANCE AND DATA SPLITS

Every result in Appendix K is bound to artifacts frozen before it. The reference edit law was never retrained for any experiment in this paper, and the controller head evaluated on the prospective panel was verified byte-identical by hash before and after an unrelated training session. The three source panels (development, prospective validation, and the official cohort) were verified pairwise disjoint, with the official cohort and the development panel excluded from the eligible pool before the training corpus was sampled. Role precedence is final test, then controller validation, then validation, then train, so a molecule appearing in two raw libraries is excluded from the lower-precedence role; the one such case is excluded from training and appears in neither the final training set nor the matched reserve.

Both evaluation panels are exactly disjoint and near-duplicate disjoint from the training source universe of 96,094 molecules. The matched reserve is cluster-disjoint by construction under an ECFP4 Tanimoto threshold of 0.95 with formula and scaffold blocking, and the external cohort shows zero exact overlap under a scaffold-matched comparison. The reserve’s higher scaffold sharing is by design: it was carved

from the same population, which is what makes it population-matched. The evaluation ladder and its no-rescue rules were recorded before the data: the validation panel is opened once, whatever it returns is banked, and no hyperparameter is re-chosen afterwards.

A pre-freeze mechanical smoke test executed one official benchmark source before the stop condition took effect. Its output was never inspected and is excluded from all analyses; the official cohort therefore remains unobserved for inferential purposes. The sealed final-test role, comprising 33,683 source records, has not been leakage-audited, that audit is a precondition for opening it, and no result in this paper draws on it.

I MODEL AND TRAINING DETAILS

Three learned objects appear in this paper and are trained separately: the goal-independent reference law R_θ , a property model $\hat{F}_\psi$ that defines molecular desirability, and goal- and budget-conditioned future-value models. Only the future-value models are trained against a downstream design objective; R_θ never sees one.

I.1 REFERENCE EDIT LAW

R_θ is fitted once, frozen, and never retrained for any experiment reported here. Its identity is pinned by the sampling-law hash recorded in the corpus binding artifact and by the dataset, library, packed-store and reserve digests alongside it. The conditional edit law is normalized over the legal edit set as in (12) and is hierarchical as in (13): a shared molecular encoder produces a state representation of width 256, a family gate scores which rewrite family fires, and family-specific scorers rank legal operands and payloads within it. Normalization is restricted to candidates the executor has admitted, so the model never places mass outside $\mathcal{A}(x)$. The encoder reads the current molecular graph and a progress variable, and does not read the source lead, the objective, the remaining budget, or any task constraint.

The reference law is trained by maximum likelihood of the next committed molecular successor under the declared editing training measure. For a productive teacher transition $x_i \rightarrow y_i^*$,

$$\mathcal{L}_{\text{ref}}(\theta) = - \frac{\sum_i w_i \log R_{\theta,t_i}(y_i^* | x_i)}{\sum_i w_i},$$

as in (14), where $R_{\theta,t}$ is the canonical successor kernel of Appendix C.3 and terminal rows are excluded. For the frozen checkpoint used throughout this paper, teacher rates and importance weights are unit-valued; the corpus sampler instead defines the reference measure through its registered family-weighted row distribution. The total event intensity is not optimized by this objective, and fixed-budget COMPOSE therefore does not claim calibrated continuous-time holding rates.

Training. Encoder depth, message passing, optimizer, schedule, batch size, optimization steps and the checkpoint-selection criterion are fixed in the frozen training configuration released with the reference-law checkpoint; no value was tuned against any reported panel.

I.2 PROPERTY MODELS

Molecular desirability is defined through a frozen property model $\hat{F}_\psi$ rather than through oracle calls at control time. For the two-objective experiment it is fitted to a matched budget of 10,000 random-ZINC property labels, the same budget the comparator receives, and is shared identically by the immediate and future-aware arms so that any difference between them is attributable to control rather than to the landscape. Both arms therefore inherit the same prediction error and the same extrapolation behaviour outside the label range.

Architecture and training. Architecture, featurization and held-out predictive performance are recorded with the released property-model checkpoints.

I.3 QED FUTURE-VALUE MODEL

The region-goal controller estimates finite-budget reachability of a goal region and is used for the similarity-constrained editing experiments. Its settings are design constants fixed in frozen configuration before any reported panel was run, and are summarized in Table 5.

Four properties of this design matter for the claims and are not conveyed by the table. Goal conditioning is continuous, so a threshold pair never seen in training is a point in the same input space rather than an unseen class label. The validation split is by source molecule rather than by training example, because

Table 5: Region-goal future-value model. All entries are frozen configuration constants, not tuned against any reported panel.

Component	Setting
Inputs	goal coordinates, signed property margins, remaining-budget one-hot, graph embedding
Head	MLP $1055 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 1$, ReLU, dropout 0.1
Output	sigmoid reachability
Loss	binary cross-entropy + $0.3 \times$ Bellman consistency
Optimizer	Adam, learning rate 10^{-3}
Goal grid	5 QED thresholds $\times$ 4 similarity floors
Validation split	0.15, held out by source molecule

a trajectory rolled out of one lead yields many near-duplicate states and splitting by example would leak them across the boundary. The boundary condition $h_0 = g_z$ is imposed exactly rather than learned. The R_θ encoder is frozen while this head is trained, so the reference process is unchanged by controller fitting.

I.4 MULTI-OBJECTIVE FUTURE-VALUE MODEL

The multi-objective controller estimates a different quantity from the region model above and is documented separately for that reason. Its target is the expected preference-weighted terminal desirability under the frozen reference process, $h_b(x, \lambda) = \mathbb{E}_{R_\theta}[g_\lambda(X_b) \mid X_0 = x]$, with g_λ the Chebyshev desirability of (27); it is a soft expectation rather than a hitting probability, so the binary construction of the region model does not carry over.

For numerical conditioning, the implementation evaluates the equivalent centered form $\tilde{g}_\lambda(x) = \exp(-(L_\lambda(x) - L_\lambda^*)/\tau_\lambda)$, where L_λ^* is a per-preference constant with respect to x . This multiplies all desirabilities for a fixed preference by the same positive constant, so the factor cancels in the normalization of (23) and leaves every controlled transition unchanged.

Supervision is Monte Carlo: M independent continuations of the frozen reference law from a state give the estimator of (22). The variance of that estimator grows with the remaining budget while the spread of true values across candidate successors shrinks, so target reliability degrades with horizon. Appendix K.5 reports that degradation and the rollout counts it implies.

Because supervision quality is horizon-dependent, learned future-value control is admitted only at horizons whose target reliability meets a criterion fixed in advance of any optimization result. The admitted horizon and its rollout count are recorded in the frozen controller configuration, and the deployed score is $h_\phi(y, \lambda, b - 1)$ for $b \leq b_{\text{FA}}$ and $g_\lambda(y)$ otherwise.

J SAMPLING AND CONTROL

We use two numerical realizations of molecular control. Direct or rejection-based sampling is used when the controlled row can be evaluated efficiently, whereas low-reference-mass objectives are handled with the sequential Monte Carlo construction below. Both retain the frozen molecular reference process and differ only in how the controlled distribution is realized computationally.

J.1 EXACT FINITE-HORIZON CONTROL

Where the set of molecular states reachable within the remaining budget can be enumerated, the backward value of (18) is computed by exact dynamic programming: initialize $h_0 = g_z$ on the enumerated terminal layer and sweep backwards, at each budget taking the expectation of the next layer’s value under R_θ restricted to the legal edit set. The controlled kernel of (19) follows by division and Theorem 3 applies without approximation. Accordingly, empirical claims of exact finite-horizon control refer only to enumerable settings in which the backward values are computed without approximation.

J.2 MOLECULAR-SCALE SAMPLING

At scale h_ϕ replaces the exact value. Because it is a state-level function taking values in $[0, 1]$, the controlled row admits a rejection realization: propose $Y \sim R_\theta(\cdot \mid x)$, execute, canonicalize, reject self-transitions, and accept with probability $h_\phi(Y, z, b - 1)$. Conditioned on acceptance the move has exactly the $R_\theta \cdot h_\phi$ law, and aliased successors aggregate correctly because the acceptance probability depends only on the resulting molecule. The proposal cap is finite, however, so when every proposal

at a state is rejected the trajectory ends; the capped procedure is therefore not an exact realization of the controlled kernel and is not described as one. Cap pressure is recorded as a diagnostic with a preregistered escalation to the sequential realization below, which changes inference only.

When the desired region carries little mass under the reference process, control is realized as a twisted Feynman–Kac particle system. Every particle proposal comes from the frozen reference law alone, and h_ϕ enters only through the incremental weight (24). Using the value to both select and weight would double-count control and sample a different process. The weight product telescopes along a trajectory to $h_0(x_H)/h_H(x_0)$, and the terminal boundary $h_0(x) = \mathbf{1}[x \in \mathcal{B}_z]$ is exact, so a learned twist changes particle allocation without changing the terminal success criterion. The operating point is frozen and no particle-count sweep is permitted: $N = 32$ particles, horizon 24, absorbing stop on entry to the goal region, and systematic resampling whenever the effective sample size $\text{ESS}(w) = (\sum_i w_i^2)^{-1}$ on normalized weights falls strictly below $N/2$. Resampling is systematic rather than multinomial, a single uniform draw positioning the whole comb. Per-transition values, per-step resampling decisions and resampling indices are recorded so the mechanical checks run offline against the artifact.

Candidate accounting follows one rule: one run returns one molecule. A source’s twenty candidates are twenty independent runs, each contributing the single molecule drawn from its terminal normalized particle measure. The particles within a run are inference state and are never counted as candidates, and intermediate states are never retrospectively harvested. A run reaching the budget without entering the region is recorded under a frozen extinction status rather than replaced. For a region goal the controller stops at the first committed molecule inside the region, an online stopping time evaluated on the realized state.

Algorithm 1 Controlled molecular sampling, sequential realization

```

1: input: source  $x_0$ ; goal  $z$ ; budget  $K$ ; frozen  $R_\theta$  and  $h_\phi$ ; particles  $N$ ; optional rule-closed predicate  $C$ 
2: initialize  $x^i \leftarrow x_0$  and  $w^i \leftarrow 1/N$  for  $i = 1, \dots, N$ ;  $b \leftarrow K$ 
3: while  $b > 0$  and some particle is unabsorbed do
4:   for each unabsorbed particle  $i$  do
5:     enumerate  $\mathcal{A}(x^i)$ ; if  $C$  is given, restrict to  $\mathcal{A}_C(x^i)$ 
6:     push forward through the executor and canonicalize to  $\mathcal{S}(x^i)$ 
7:     draw  $y^i \sim R_\theta(\cdot \mid x^i)$  {proposal from the reference law alone}
8:      $w^i \leftarrow w^i \cdot h_\phi(y^i, z, b - 1) / h_\phi(x^i, z, b)$  {the value enters only here}
9:      $x^i \leftarrow y^i$ ; if  $x^i \in \mathcal{B}_z$  then absorb particle  $i$ 
10:   end for
11:   normalize  $w$ 
12:   if  $\text{ESS}(w) < N/2$  then
13:     systematically resample the particles and reset  $w^i \leftarrow 1/N$ 
14:   end if
15:    $b \leftarrow b - 1$ 
16: end while
17: return one molecule drawn from the terminal normalized particle measure, or the frozen extinction status if no particle was absorbed

```

J.3 MULTI-OBJECTIVE CONTROL

Under a preference λ the immediate arm weights successors by $g_\lambda(y)$ and the future-aware arm by $h_\phi(y, \lambda, b - 1)$, with the reference law, legal edit set, property model, sources, preferences and budget identical between them. Where the admitted horizon b_{FA} is shorter than the optimization campaign, a campaign consists of repeated controlled trajectories of length at most b_{FA} launched from the frozen initialization bank and its archive until the common evaluation budget is exhausted. The campaign horizon and the horizon over which learned foresight is claimed are therefore distinct quantities.

J.4 RETARGETING AND PATHWISE CONSTRAINTS

Retargeting replaces the goal context at a realized state and continues with the remaining budget by re-evaluating (23) at that state; no parameter of the reference process changes and the prefix is retained. A rule-closed constraint is imposed by filtering the legal edit set to $\mathcal{A}_C(x)$ of (28) before the transition is sampled, so no infeasible successor is proposed, weighted, or committed.

K EXPERIMENTAL DETAILS

We provide the full protocols, comparison arms, statistical units, and extended results for the experiments summarized in the main text.

K.1 REFERENCE-LAW EVALUATION

We evaluate the frozen reference law on held-out transitions from the matched reserve by measuring the probability assigned to the realized canonical successor. The uniform, empirical-family and learned laws are evaluated on the identical executable support and canonical successor quotient; only their probability assignments differ. The empirical-family law uses corpus-level edit-family frequencies renormalized over the families available at the current state and distributes each family’s mass uniformly across its available successors. Two intermediate arms decompose the comparison: an empirical family with learned identity, which keeps corpus family frequencies and learns only which successor within the chosen family is realized, and a learned family with uniform identity, which learns only which family fires. Together they separate choosing which edit to make from choosing where to apply it.

The primary quantity is the mean negative log-probability of the realized successor, with the scored transition as the unit and paired differences by construction, since every law scores the same transition. Cells below a preregistered minimum size of 20 are not reported separately; no scored transition was skipped and no partition failed to compile. Performance against the uniform law tests whether learning improves on executable support alone, and performance against the empirical-family law tests whether the reference law captures state-dependent preference beyond global family frequency. Neither comparison is interpreted in terms of physical molecular kinetics or synthetic-route likelihood.

Capability cell	<i>n</i>	unif.	emp. fam.	<i>emp+id</i>	<i>fam+unif</i>	<i>R_θ</i>
atom_restate: element identity change	725	6.21	6.54	4.84	7.28	5.58
atom_delete: leaf death	601	6.20	3.36	2.68	3.69	3.01
atom_insert: one-neighbor birth	572	6.14	7.12	5.23	6.97	5.09
bond_reroute: single-atom pendant, cyclic	464	6.24	5.26	4.07	5.48	4.29
bond_reroute: multi-atom pendant, cyclic	216	6.32	5.56	4.38	6.14	4.96
ring_system_restate: dearomatization [†]	202	6.25	2.84	2.97	1.72	1.85
ring_system_restate: aromatization [†]	165	6.51	2.85	2.34	1.12	0.61
cycle_insert: close to monocyclic [†]	138	6.47	7.19	3.27	4.80	0.88
cycle_attach: open from monocyclic [†]	104	6.16	5.24	5.20	3.34	3.29
bond_reorder: bond-order decrease	83	6.32	4.30	4.01	3.18	2.89
bond_reorder: bond-order increase	83	6.49	4.71	4.40	3.29	2.99
cycle_insert: close to polycyclic [†]	81	6.55	7.37	3.95	5.26	1.84
cycle_attach: open from polycyclic [†]	69	6.42	5.82	5.71	4.08	3.96
Overall	3545	6.25	5.37	4.10	5.17	3.90

Table 6: Negative log-probability of the realized canonical successor, by capability cell, lower is better. All five laws are evaluated on the same executable support and the same canonical quotient; only the probability assignment differs. Columns *emp+id* and *fam+unif* are the two decomposition arms defined above: empirical family with learned identity, and learned family with uniform identity. Cells below the preregistered minimum size of 20 are not reported separately; no scored transition was skipped and no partition failed to compile. [†] Supervision for these cells comes entirely from the synthetic-perturbation lane; see the provenance note below.

The decomposition is strongly asymmetric. Learning only which family fires recovers little, improving on corpus family frequencies by 0.20 nats overall, whereas learning only where within a family the edit lands recovers most of the effect at 4.10; the full model adds a further 0.20 beyond that. In the terms used in Appendix K.1, the empirical-family-to-full improvement is 1.4616 nats and the identity-only arm recovers 1.2674 of it, or 86.7%, against 13.3% for the family gate alone. The reference law’s aggregate advantage is therefore driven primarily by state-dependent successor identity rather than by the family gate. The empirical-family control is also not uniformly strong: on both *cycle_insert* cells, on *atom_insert* and on *atom_restate* it is worse than assigning equal probability to every distinct successor, so on birth and insertion families a corpus frequency prior misallocates mass relative to ignorance. On four cells the empirical-family/learned-identity ablation assigns higher probability to the realized successor than the full model, indicating that the learned family gate does not uniformly improve over corpus-level family frequencies and that the identity model carries the result on those families.

Aggregating by supervision provenance sharpens this. On the 2,744 transitions from families with data-backed supervision, the reference law retains a 1.06-nat advantage over empirical family frequencies (5.55 against 4.49), so the main-text claim survives the obvious confound. On that subset, however, the

empirical-family/learned-identity ablation reaches 4.25 nats and therefore assigns higher probability to the realized successor than the full model does, and the learned-family/uniform-identity arm at 5.79 is worse than corpus family frequency. The learned family gate is thus not uniformly beneficial, and on data-backed families the identity model carries the result in spite of it. The complementary synthetic-only subset comprises 759 transitions, where the reference law reaches 1.79 nats against 4.72 for empirical family frequencies; the aggregate improvement reported in Appendix K.1 is correspondingly larger than the data-backed figure. The partition is computed from the emitted family-lane provenance artifact rather than from a hand-specified family list, and the restricted aggregates are reproduced from it.

The cells in which the reference law performs best are those whose supervision is synthetic, as recorded in Appendix H. On the ring and cycle families the model recovers a perturbation of a kind it was trained to invert, and the aggregate improvement should not be read as evidence about molecular kinetics. Resolving instead by the number of available successors gives 4.86 nats for states with no alternative in the recorded band and 5.51, 5.44 and 5.51 for successor counts of 1–4, 5–24 and 25 or more under the empirical-family law, against 4.34, 4.03, 4.00 and 4.21 for the learned-identity arm, so the learned advantage is not an artifact of nearly deterministic rows.

K.2 FUTURE-REACHABILITY EVALUATION

The panel was sealed under preregistration and opened once. Sixty-seven held-out source–target pairs were constructed whose target is reachable under the frozen executable process within the declared budget; two were excluded from the primary analysis under a committed amendment, giving 65 pairs, and the full 67 are retained as a sensitivity analysis. All arms share the reference law, the legal edit sets, the target molecule and the edit budget. A myopic policy selects the locally preferred edit; verified lookahead evaluates candidate successors by whether the target remains recoverable within the remaining budget; three intermediate prioritizers restrict which successors are expanded before lookahead is applied. Continuation work is reported as a fraction of the candidates evaluated by all-candidate lookahead, so recovery and cost are read together. The unit is the source–target pair and intervals are paired bootstrap.

Arm	Rec.	Rate	Cont./pair	Frac. univ.	Gain ret.
Greedy local decision	26/65	40.0%	0.0	0%	—
R_θ top-1 + lookahead	33/65	50.8%	7.3	34.7%	0.50
Similarity top-1 + lookahead	34/65	52.3%	7.3	34.7%	0.57
Similarity top-2 + lookahead	37/65	56.9%	10.8	52.1%	0.79
Goal-aware h_ϕ top-1 + lookahead	40/65	61.5%	7.2	34.6%	1.00
All-candidate lookahead	40/65	61.5%	20.5	100%	1.00

Table 7: Sealed panel, 65 pairs. *Rec.*: targets recovered. *Cont./pair*: candidate continuations actually evaluated per pair. *Frac. univ.*: that count as a share of what all-candidate lookahead evaluates. *Gain ret.*: the fraction of all-candidate lookahead’s 14 rescues the arm also recovers.

Of the 65 pairs, 14 are recovered by all-candidate lookahead and not by greedy, and none by greedy and not by all-candidate lookahead. Because the planner retains the greedy action unless lookahead gives a strict improvement, the comparison is one-sided by construction. We therefore report the paired recovery gain, 21.5 percentage points [12.3, 33.5], and the fraction of greedy’s 39 failures that lookahead rescues, 35.9% [21.2, 52.8], rather than a discordance test.

Equal recovery counts do not imply equal rescue sets. The goal-aware prioritizer and all-candidate lookahead both recover 40 targets but not the same 40, overlapping on 13 of 14 rescues; the similarity and reference-probability arms are strictly nested inside all-candidate lookahead’s rescue set. Recovery was also resolved by verified path length: the advantage does not widen with path length, and the original hypothesis that it would was tested and not supported. As a sensitivity check, including both pairs excluded under the committed amendment gives 26/67 for greedy and 41/67 for all-candidate lookahead, a headroom of 15 rather than 14, with no ordering changes. The panel consumed 1,347 kernel calls.

K.3 SOURCE-CONDITIONED MOLECULAR EDITING

The protocol is adopted from the comparator and is given in Appendix L.1. Each COMPOSE candidate is the output of one controlled trajectory terminating prospectively at the first realized state inside the goal region, and intermediate molecules are not retrospectively collected. The reference law is unchanged from Appendix K.1, and the future-value controller is trained and frozen on data disjoint from the

official sources before the cohort is opened. A size-fixed internal ablation removes every successor that changes the current heavy-atom count while holding controller, reference law, sources, seeds and candidate allowance fixed. The unit is the source molecule; the candidate curve is descriptive and carries no significance claim, and it is drawn only over the k actually measured, which is $k = 1, \dots, 8$ on the official cohort. The official cohort is opened once and the result is reported as returned.

A receding one-step terminal tilt of the frozen reference law by the immediate property value of each successor was preregistered as the inference policy and refuted on the disjoint development panel, with 0 successes in 320 trajectories, before any official-cohort outcome existed. It is retained as a developmental negative control. What this licenses is narrow: the finite-horizon controller is a pre-outcome amendment consistent with the paper’s construction, motivated by a developmental failure of a cheap one-step approximation. It is not a restoration of an originally frozen policy, and compute cost was not the only consideration separating the two.

On the official cohort, COMPOSE solves 446 of the 800 sources at $K = 8$ (55.8%, 95% Wilson interval [52.3, 59.2]), against 45.1% reported by GrIDDD at $K = 20$. Two sources did not return a completed shard and are counted as failures, so the denominator is the full benchmark.

K.4 SIMILARITY-CONSTRAINED LEAD OPTIMIZATION

Panel and constraints. The panel is five protein targets (PARP1, FA7, 5HT1B, BRAF, JAK2), three seed molecules per target, and two similarity thresholds $\delta \in \{0.4, 0.6\}$, giving 30 cells. A returned molecule is feasible when $\text{QED} \geq 0.6$, $\text{SA} \leq 4$, and $\text{Tanimoto}(x, x_0) \geq \delta$ against the cell’s seed, with similarity on Morgan fingerprints of radius two. Among feasible molecules the objective is the QuickVina2 docking score, lower being better. These are endpoint constraints only: intermediate molecular states are not required to satisfy them, which is what makes the trajectory free to leave and re-enter the feasible region.

Frozen process and controller provenance. The executor and the reference law are the same frozen artifacts used throughout: R_θ is the GuacaMol-trained checkpoint of Appendix I, not retrained or fine-tuned for this benchmark. Only the task-specific controller differs, and the docking-facing controller was frozen before the reported panel was launched. Feasibility predicates (QED, SA, similarity, chemical validity) are evaluated before the protein oracle, so docking calls are spent only on molecules already satisfying the benchmark constraints.

Oracle budget and reported score. Each reported COMPOSE entry consumes 500 docking-oracle evaluations for its cell, and the score reported for that cell is the best feasible docking score obtained within that budget. The published GenMol entry corresponds to 1,000 oracle calls per run averaged over three runs, i.e. 3,000 calls per cell, so the comparison is made at a sixfold reduction in oracle budget. Budget is counted as docking calls; inexpensive property evaluations are not counted for either method, matching the comparator’s accounting.

Feasibility and the paired comparison. A cell counts as feasible for a method when that method reports at least one molecule satisfying all three constraints. The paired comparison is restricted to cells where both methods are feasible, and the estimand is the mean per-cell difference in docking score, COMPOSE minus GenMol, with a 95% interval over cells. Cells where either method reports no feasible molecule enter the coverage count but not the paired mean, and are never imputed.

Docking replicate noise. Repeat evaluation of the same molecule through our QuickVina2 pipeline gives a median replicate variation of 0.70 kcal/mol. Per-cell differences smaller than this are not individually resolvable, which is why the paired aggregate rather than the win count is the reported comparison, and why cell-level differences are not interpreted separately.

Comparator values. GenMol, RetMol and Graph GA entries are the published values of Table 4 of Lee et al. (2025), transcribed without re-derivation. The transcription was verified by reproducing the GenMol column cell-by-cell against an independently extracted copy of the same table; all 30 cells agree, which fixes the row order and column alignment for the RetMol and Graph GA columns as well. Dashes in that table denote cells for which no feasible solution is reported, and are carried through as missing rather than as zeros.

K.5 FUTURE-VALUE TARGET RELIABILITY

Before training a future-value model, we measured how well its Monte Carlo target can be estimated. On 24 held-out states with 16 candidate successors each and 32 reference continuations per candidate,

scored by the frozen property model with no oracle calls, the rank correlation between immediate desirability and true future value has per-state median 0.518 [0.435, 0.700] at four remaining edits and 0.318 [0.181, 0.421] at eight, with intervals resampled over states. The two criteria select different successors on 68.3% [55.8, 80.8] and 84.2% [73.3, 93.3] of state–preference decisions respectively, and following future value rather than immediate desirability improves realized continuation value by a median 12.2% [1.6, 19.8] at four edits and 19.0% [13.1, 25.9] at eight.

Target reliability itself degrades with horizon. Estimated by split-half agreement between disjoint sets of continuations, a 32-rollout target has reliability approximately 0.66 at four remaining edits and 0.39 at eight, implying by the Spearman–Brown relation that roughly 65 and 200 continuations respectively would be required to reach a reliability of 0.80. Because supervision quality is horizon-dependent while the measured benefit is not, the supported horizon is fixed from target reliability and computational feasibility rather than from optimization performance, as recorded in Appendix I.

K.6 MID-TRAJECTORY RETARGETING

We evaluate retargeting on 40 source-disjoint held-out molecules under potency-first and developability-first histories. Each prefix contains three committed edits within a six-edit horizon and is fixed before the eventual joint objective B is constructed, so every post-switch comparison begins from an identical realized prefix with the same remaining budget. Margins are normalized by the held-in interquartile range and the reported objective is the worst margin of the conjunction, with zero denoting satisfaction. At the switch the median worst margin is -1.115 following potency-first prefixes and -1.810 following developability-first prefixes, so neither history already satisfies B .

Comparison	History	Mean	Median	95% CI	W/L
Responsiveness	potency	+0.748	+0.794	[+0.608, +0.891]	36/0
	developability	+1.462	+1.357	[+1.240, +1.691]	40/0
Greedy-matched history	potency	+0.367	+0.338	[+0.251, +0.482]	37/3
	developability	+0.211	+0.147	[+0.065, +0.361]	26/14
<i>Value of history</i> (primary)	potency	+0.302	+0.243	[+0.186, +0.413]	35/5
	developability	+0.176	+0.127	[+0.069, +0.283]	29/11
Price of surprise	potency	-0.252	-0.123	[-0.364, -0.155]	6/32
	developability	-0.428	-0.291	[-0.580, -0.288]	4/36

Table 8: Held-out retargeting panel, 40 source-disjoint sources per history. Rows compare, in order: retargeting against continuing the old goal; greedy retargeting against greedy restart from the source; verified continuation from x_3 against verified restart from x_0 (the primary estimand); and verified continuation against a clairvoyant controller given B from the first edit. Entries are paired post-switch differences in the registered objective; positive values favor the first strategy named. W/L counts sources on which that strategy is better or worse; ties omitted. Intervals are source-level bootstrap.

Retargeting materially redirects both histories, and under the primary verified comparison continuing from the realized state remains preferable to restarting from the source. The mean advantage is +0.302 [0.186, 0.413] following potency-first prefixes and +0.176 [0.069, 0.283] following developability-first prefixes, with wins on 35/40 and 29/40 sources respectively. The clairvoyant controller remains better, giving the negative price-of-surprise effects shown in the table. The effect is smaller than in development, with the primary differences shrinking from +0.444 to +0.302 and from +0.290 to +0.176, and the confirmation panel contains genuine source-level losses in both histories, so the benefit of state reuse is empirical rather than guaranteed. Prefix revisiting is negligible, occurring for at most one source per arm and history, and does not explain the effect.

K.7 PATHWISE CONSTRAINTS

The pathwise experiment remains developmental: the cLogP corridor was selected following an earlier feasibility screen, and a separately frozen held-out confirmation has therefore been specified but not opened. Endpoint-only control constrains only the returned molecule; pathwise control removes any successor outside the corridor before the transition is sampled; a terminal-penalty arm changes the objective but leaves the transition support unchanged. All arms share the reference law, objective, horizon, sources, seeds and evaluation budget, and the unit is the source molecule with source-clustered intervals.

On 24 eligible sources, endpoint-only trajectories return to the corridor after leaving it in $16/24 = 0.667$ [0.458, 0.833] cases under greedy control and $14/24 = 0.583$ [0.375, 0.792] under verified control; the

conditional fractions are 16/24 and 14/23, so the conditional denominators remain large enough that the unconditional and conditional prevalence estimates agree closely. These excursions are not confined to the boundary: their median depth is 0.69 cLogP units and their median duration is four of the six committed edits. An earlier protected-motif experiment produced zero such excursions, confirming that the estimand need not be positive simply because intermediate states are observed.

Restricting the support to the corridor remains viable on this panel. The median source retains 79.8% of its legal successors, only one source reaches the prespecified support-tight threshold, no evaluated state has an empty constrained edit set, and one trajectory terminates. Excluding the support-tight source leaves prevalence at 0.652 and 0.565 and terminal costs at -0.121 and -0.041 , so the conclusions are unchanged.

The developmental panel does not resolve the terminal-utility cost of imposing the constraint. The paired change in normalized DRD2 utility is -0.105 $[-0.377, 0.099]$ under greedy parity, with 5 wins, 8 losses and 11 ties, and -0.023 $[-0.182, 0.134]$ under verified parity, with 9 wins, 10 losses and 5 ties. Both medians are exactly zero and both intervals span zero, which does not establish equivalence: the greedy interval still permits a loss of 0.377 held-in interquartile-range units. Zero violations under support restriction is an integrity consequence of masking rather than an empirical performance statistic, and the apparent advantage of future-aware control under the mask, $+0.661$ $[0.428, 0.944]$, is a ceiling rather than a null, since greedy pathwise control completed all 24 sources and left no binary headroom.

Constraint handling	Endpoint-feasible runs hiding a path-infeasible intermediate	Paired terminal DRD2 utility
Endpoint filtering (greedy)	16/24 [45.8, 83.3]	reference
Endpoint filtering (verified)	14/24 [37.5, 79.2]	reference
Support restriction	0 by construction	-0.023 $[-0.182, 0.134]$

Table 9: Constraint handling on the 24-source developmental panel. The first column counts sources whose returned molecule satisfies the corridor while an intermediate state does not; support restriction excludes these by construction rather than by measurement. The utility column is the paired change against endpoint filtering under verified parity, with the greedy-parity value -0.105 $[-0.377, 0.099]$. Both intervals span zero, so the panel bounds rather than resolves the terminal-utility cost.

The held-out confirmation fixes the corridor at $C = [2.3689, 4.4522]$ with the same six-edit horizon and endpoint-only semantics, at $n = 48$ sources. It passes only if both criteria clear at full α under intersection-union: the lower bound of the source-clustered interval on hidden-path prevalence must exceed $1/3$, a threshold fixed before the development run, and the lower bound on the mean paired change in terminal utility must exceed -0.25 held-in interquartile-range units, with -0.20 as a prespecified sensitivity analysis. The panel is sized on the joint criterion, giving power 0.976 at the 0.25 margin and 0.874 at 0.20. No held-out source has been opened.

A value head retrained to be aware of a longer horizon was built, evaluated and rejected on four separate measurements, and is not the head used anywhere in Appendix K. The consequence for the main text is a claim boundary: any gain from a longer horizon comes from giving the frozen process more edit opportunities, not from a better budget-conditioned value model.

L BENCHMARK AND BASELINE DETAILS

L.1 QED EDITING BENCHMARK

We adopt the official 800-source cohort, property and similarity criteria, and evaluator from the benchmark, while evaluating COMPOSE with $K = 8$ rather than the published $K = 20$ allowance: the 800 test molecules with $\text{QED} \in [0.7, 0.8]$ as sources, 20 candidates per source, a Tanimoto similarity constraint of 0.4 to the source, and success defined as at least one returned candidate reaching $\text{QED} \in [0.9, 1.0]$ while satisfying that constraint. Source selection, fingerprints, property calculations, candidate count and aggregation must be identical for the comparison to stand, and where any of these could not be matched it is stated rather than assumed.

Output contract. Each source receives exactly K candidate slots. A trajectory that reaches the target region returns the first committed molecule satisfying the benchmark criterion. If no sampled continuation reaches the region, the slot returns the unchanged canonical source solely to preserve the fixed- K benchmark interface; this output necessarily fails because benchmark sources begin below the

target QED threshold. No failed slot is resampled or replaced. This contract is frozen before the official cohort is opened and is not revised afterwards.

Every non-source returned molecule is a benchmark success: each slot either returns a molecule in the target region or returns the unmodified source. A smaller candidate allowance can disadvantage both success and the diversity metric, because a source with only one successful distinct return contributes zero diversity. Separately, no returned success is identical to its source, so the count is unchanged under the stricter rule of the released success script, which requires similarity strictly below one.

Published values are reproduced as reported and are not re-derived. Because the comparator does not fully specify the evaluator used for its main diversity entry, we do not interpret the diversity values as an exactly matched comparison. Among published entries for this task, the GrIDDD row at 45.1% is the one we compare against. Higher rates have been reported on this task outside the comparator’s table, and Appendix L.3 records them so that Table 10 is not read as the complete published record.

Diversity is computed with the released evaluation script of Jin et al. (2019) rather than reimplemented from its prose description. That script admits a candidate only if it satisfies both the similarity constraint and the property target, deduplicates the admitted molecules, excludes a source with no admitted molecule from the average, and assigns a diversity of exactly zero to a source with only one. Similarity is a Morgan fingerprint of radius two on 2048 bits with chirality disabled. The convention matters. Of the 446 solved sources on the official panel, 335 return at least two distinct non-source molecules and average 0.4999; the remaining 111 return a single distinct molecule and enter the released script’s average as zero, giving 0.376 over all solved sources. We report the 0.4999 figure and state its denominator wherever it appears. COMPOSE returns on average 3.63 distinct successful molecules of the eight permitted, and no successful candidate is identical to its source, so the success count is unchanged under the stricter rule of the released script, which requires similarity strictly below one. All returned strings are canonical, so the string-level deduplication of the released script coincides with deduplication by molecule, and no pair of distinct returned molecules shares a fingerprint under the achiral setting.

GrIDDD does not specify the evaluator used for the diversity entry in its main property-optimization table. Ninniri et al. (2025) do not state how the diversity column of their main property-optimization table is computed. Their baseline diversity values are identical to those reported in the graph-to-graph translation line, which places the inherited rows on the scale defined above, but the method’s own entry is undocumented; a rule matching the one used here is stated only in their appendix, for a separately adapted protocol with a different number of optimization rounds. We therefore report our value as computed under a named and audited definition, and we do not assert that it and the published entry were produced by the same procedure.

Because every COMPOSE trajectory terminates prospectively on entering the goal region, a run either returns a molecule satisfying both constraints or goes extinct, and the set the diversity script admits is exactly the set of successful returns. The admitted set is therefore constructed the same way as in the published rows rather than merely being similar to it. Both now use the official 800 sources, and the residual difference is the candidate allowance, eight against twenty. That difference is not neutral for diversity: a smaller allowance yields fewer second successes per source and so more single-molecule sources contributing zero, which depresses the reported value relative to what the same system would produce at twenty candidates.

The benchmark constrains returned molecules and says nothing about internal successor evaluations, and competing methods use their native inference procedures. The comparison therefore measures task performance under a fixed output allowance, no efficiency claim is made, and inference ledgers are reported separately. The printed entry is the official 800-source cohort at eight of the twenty permitted candidates, so it uses fewer outputs than the benchmark allows.

L.2 GENMOL T4 LEAD-OPTIMIZATION BENCHMARK

The protocol is adopted from Lee et al. (2025): five protein targets, three seed molecules per target, and similarity thresholds $\delta \in \{0.4, 0.6\}$, with a returned molecule required to satisfy $\text{QED} \geq 0.6$, $\text{SA} \leq 4$, and Tanimoto similarity at least δ to the seed. The reported statistic in the comparator’s table is the mean docking score of the most optimized lead over three runs. The COMPOSE entry for a cell is the best feasible docking score obtained within that cell’s 500-call budget.

Two protocol details govern how the comparison is read. First, the constraints are endpoint constraints: neither method is required to keep intermediate states feasible, so the comparison is about returned molecules only. Second, oracle budget is counted in docking calls. The comparator’s entry corresponds

to 1,000 calls per run over three runs; the COMPOSE entry consumes 500 calls for the cell. Inexpensive property evaluations are excluded from the count on both sides.

Dashes in the published table mark cells for which no feasible molecule is reported. We carry them through as missing rather than as zeros, report coverage separately from score, and restrict the paired comparison to cells where both methods report a feasible molecule. Published values for GenMol, RetMol and Graph GA are reproduced as reported and are not re-derived.

L.3 PUBLISHED RECORD ON THE QED EDITING TASK

Method	Sources	K	QED success ($\uparrow$)	Diversity ($\uparrow$)
GrIDDD (Ninniri et al., 2025)	800	20	45.1%	0.283 [†]
COMPOSE	800	8	55.8%	0.4999

Table 10: **Source-conditioned editing on the Jin/ZINC-250k similarity-constrained QED benchmark.** Published baseline values are reproduced from the comparator set reported by GrIDDD. K denotes candidates returned per source; success requires QED ≥ 0.9 and Tanimoto similarity ≥ 0.4 . COMPOSE is evaluated on the official 800 sources with $K = 8$; published rows use $K = 20$, so the comparison is at a smaller candidate allowance. [†]GrIDDD reports diversity without specifying its evaluator implementation; COMPOSE uses the released Jin evaluator. Diversity details are reported in Appendix L. GrIDDD is shown as the close modern per-step editor that motivates this benchmark; the Jin-era published methods and the full comparator audit are given in Appendix L.3.

Table 10 reproduces the comparator set assembled by GrIDDD. Stronger values have been reported on the same task by the graph-to-graph translation line, and we record them in Table 11 for completeness. We checked the primary sources rather than inheriting the numbers second-hand. The translation results use the same 800 source molecules, the same similarity threshold $\delta = 0.4$, and the same allowance of $K = 20$ decoded candidates per source (Jin et al., 2019); the rate frequently quoted as 60.6% is VJTNN+GAN, whereas VJTNN itself is reported at 59.9%. Jin et al. (2020) evaluate on the test sets provided by Jin et al. (2019) under the same range and similarity constraint with $K = 20$, and reproduce the 59.9% row, which is what places their 76.9% on the same cohort.

Method	Reported by	K	QED success ($\uparrow$)	Diversity ($\uparrow$)
MMPA	Jin et al. (2019)	20	32.9%	0.236
VSeq2Seq	Jin et al. (2019)	20	58.5%	0.331
VJTNN	Jin et al. (2019)	20	59.9%	0.373
VJTNN+GAN	Jin et al. (2019)	20	60.6%	0.376
AtomG2G	Jin et al. (2020)	20	73.6%	0.421
HierG2G	Jin et al. (2020)	20	76.9%	0.477

Table 11: **Further published results on the same similarity-constrained QED editing task.** All rows use the 800-source cohort of Jin et al. (2019) with $\delta = 0.4$ and $K = 20$. These are supervised graph-to-graph translation models trained on parallel source-target pairs curated for the target property, whereas the COMPOSE reference law is trained once without reference to QED and the property enters only through control. The rows therefore measure different problem settings and we do not present them as an ablation of one another, nor is any COMPOSE claim stated relative to them.

M SOFTWARE, RANDOMIZATION, AND COMPUTE

M.1 SOFTWARE AND ENVIRONMENT

Every job runs in a containerized Debian image pinned to Python 3.11 with torch 2.4.0, numpy 1.26.4, scipy 1.13.1 and rdkit 2024.3.5. Chemistry is pinned to a single RDKit release because canonical molecular identity, ring perception and sanitization all come from it and all three enter the definition of the state space in Appendix F.

Only artifacts satisfying the evidence criteria used in the manuscript are cited for scientific claims. Developmental artifacts lacking complete producer or input metadata are retained for audit purposes but excluded from claim-bearing results. Bulk artifacts such as corpora, embeddings, per-run records and trained heads live on a persistent volume rather than in the source repository, while result summaries are versioned alongside the code, so a fresh checkout can re-run every job but cannot read a banked result without the volume. The reproducibility package records the code revision, input identities and result summaries required to regenerate each reported analysis.

M.2 RANDOMIZATION AND STATISTICS

Run seeds are derived rather than drawn, by hashing a protocol tag together with the source molecule and the replicate index and taking the leading eight bytes of the digest. In paired comparisons the arm name is deliberately excluded from the seed, so arms sharing a source and replicate index consume identical randomness and a difference between them cannot be a seeding artifact. Where an arm-specific seed is unavoidable for later replicates, the first replicate is still constructed to be identical across arms.

Intervals are bootstrap intervals resampled over the independent unit named in each experiment. That unit is the source molecule in most cases, the source–target pair in the sealed lookahead panel, and the state in the reliability analysis of Appendix K.5. Replicate counts differ by analysis, ranging from 4,000 to 8,192 draws, and each artifact records its own. Where an interval is exact rather than bootstrapped, that is stated with the reason.

M.3 COMPUTE

Reference-law training used a preemptible A10G GPU with 8 CPU cores and 64 GiB of memory; the learned future-value models were trained on the same GPU class. The claim-bearing molecular sampling and evaluation jobs reported in Appendix K are CPU-based. Training and inference resources are therefore reported separately. Sampling containers reserve one core and burst to eight under a six-hour ceiling, and driver containers hold a quarter core under a twelve-hour ceiling. Compute is reported in two units that are never pooled: kernel calls, meaning evaluations of the learned reference law, and container-seconds. The sealed lookahead panel consumed 1,347 kernel calls and roughly 24,300 container-seconds, and retargeting branch points cost on the order of 10^2 kernel calls and 10^3 container-seconds each. Oracle calls are counted separately again, because the benchmark budgets in Appendix L constrain returned molecules rather than internal evaluations.

M.4 THE MEDICINAL-CHEMISTRY VALIDITY GATE

The gate rejects a returned molecule for a small, explicitly enumerated set of reasons: elements outside {C,N,O,F,S,Cl,Br,I,P}; a halogen carrying hydrogens or more than one heavy neighbour (hypervalent iodine such as [IH4]); sulfur carrying hydrogens with two or more heavy neighbours; phosphorus that is neither phosphate nor phosphonate; radicals; formal charge beyond ± 1 ; a ring larger than seven whose atoms are mostly *not* shared with another ring; a nitrogen–nitrogen bond inside a three- or four-membered ring; and cumulated double bonds.

Two of these conditions were narrowed after calibration refuted a broader version. A flat ring-size cutoff was rejected because the benchmark seeds themselves report 7- and 8-membered SSSR envelope rings over fused tricyclic cores, so ring size alone cannot separate a fused drug scaffold from a macrocycle; the condition now applies only to rings whose atoms are largely unshared. A minimum-ring-size condition for pendant systems was rejected because a published InVirtuoGen winner carries a cyclopropane; that minimum was moved out of the validity gate and into the definition of the specific construction operation it belongs to.

The calibration is a test rather than a claim: the gate must accept all fifteen benchmark seeds and all twenty-five published InVirtuoGen winners, and reject the implausible molecules. The suite also asserts that ordinary sulfur chemistry (thiophene, thioether, sulfone) still passes, which is what prevents the hypervalent-sulfur condition from over-rejecting.

We distinguish two gates. A *pathwise* gate asks only whether a state is an executable molecule and is applied to every intermediate; the full gate, which additionally imposes QED, SA and similarity, is applied only to returned molecules. Conflating them is not conservative: applying the full gate to every intermediate prevents a trajectory from passing *through* a temporarily unattractive molecule, and measurably degraded search.

M.5 GENERALIZATION OF A SINGLE-CELL CONTROLLER

The lead-optimization controller described in Table 1 was developed against one cell. Applied unchanged to three further cells it returned no constraint-satisfying molecule from three hundred trajectories each. The failure is quantitative rather than mysterious: the similarity budget is a property of the seed, and the seed the controller was tuned on carries 41 Morgan-fingerprint bits while another carries 29, so an edit program calibrated on the first spends a much larger fraction of the second.

Replacing the fixed program with an adaptive search over the same action vocabulary — identical operators, identical reference law, no new chemistry — recovered feasibility on all three cells. We report

this as a limitation of fixed edit programs rather than as a positive result, since the recovered molecules were not competitive; but it does locate the failure in the controller's lack of adaptation rather than in the operator set or the reference law.